Unlocking the Potential of Operations Research for Multi-Graph Matching

Max Kahl[*1], Sebastian Stricker[*1], Lisa Hutschenreiter[1], Florian Bernard[2], and Bogdan Savchynskyy[1]

[1]Heidelberg University
[2]University of Bonn

July 2, 2024

Abstract

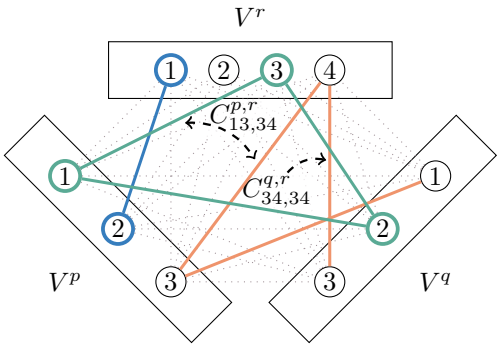
We consider the incomplete multi-graph matching problem, which is a generalization of the NP-hard quadratic assignment problem for matching multiple finite sets. Multi-graph matching plays a central role in computer vision, *e.g.*, for matching images or shapes, so that a number of dedicated optimization techniques have been proposed. While the closely related NP-hard multi-dimensional assignment problem (MDAP) has been studied for decades in the operations research community, it only considers *complete matchings* and has a *different cost structure*. We bridge this gap and transfer well-known approximation algorithms for the MDAP to incomplete multi-graph matching. To this end, we revisit respective algorithms, adapt them to incomplete multi-graph matching, and propose their extended and parallelized versions. Our experimental validation shows that our new method substantially outperforms the previous state of the art in terms of objective and runtime. Our algorithm matches, for example, 29 images with more than 500 keypoints each in less than two minutes, whereas the fastest considered competitor requires at least half an hour while producing far worse results.

1 Introduction

Establishing correspondences across multiple finite sets is a fundamental combinatorial problem that has long played a central role in many computer vision and other applications such as 3D model retrieval [33], shape matching [41, 19], statistical shape modeling [48, 21], federated learning [30], and genomic data analysis [13]. Typically, keypoints of semantically similar images, shapes, or other *objects* are matched. These matchings must satisfy several practically motivated conditions, see Fig. 1:

- *Uniqueness.* Each keypoint of a given object can be matched to *at most* one keypoint of any other object. If "*at most*" is substituted with "*exactly*", one speaks of a *complete* and otherwise of an *incomplete* matching. Due to occlusions and noise during the keypoint extraction, the incomplete formulation is preferred in practice.
- *Cycle consistency.* If keypoint 1 in object V^p is matched to keypoint 2 in object V^q and keypoint 3 in object V^r, then keypoint 2 must be matched to keypoint 3. Similar transitivity conditions must hold for all matching cycles across arbitrary object subsets.
- *Costs.* Matchings must be *minimal* w.r.t. given costs, quantifying the similarity between all keypoints. Costs for d objects decompose into a sum of costs for each of the $d(d-1)/2$ object pairs. The latter, in turn, are sums of *linear*, keypoint-to-keypoint costs and *quadratic*, keypoint-pair-to-keypoint-pair costs. Quadratic costs allow the modeling of mutual geometric relations between keypoints, considerably improving matching accuracy [20].

[*]The two authors contributed equally to this paper.



Shown are three objects (solid rectangles) V^p, V^q, V^r, whose keypoints (circles) must be matched (edges). The blue and green matchings are cycle-consistent and form cliques in the shown 3-partite graph. The orange matching is not cycle-consistent and as such does not form a clique. Examples of linear and quadratic costs (dashed arrows) and their associated matchings are also shown.

Figure 1: **Multi-Graph Matching and Cycle Consistency.**

Table 1: MGM and MDAP algorithms with a similar structure to ours, chronologically ordered. See the main text for more comments, references and contribution details. **#GM-calls** - number of calls to a GM solver to construct an initial feasible solution utilizing the whole cost vector; **GM-LS** - contains GM local search; **Swap-LS** - contains swap local search, with *single* or *mutiple* swaps; **costs** - only linear or also quadratic; **incompl.** - applicable to incomplete problems.

Algorithm	**#GM-calls**	**GM-LS**	**Swap-LS**	**costs**	**incompl.**
MDAP local search [5]	d	✓	single	linear	✗
joint opt. [51]	d	✓	✗		✗
affinity opt. [49]	d^2	✗	✗	quad.	✗
alternating opt. [52]	d^2	✓	✗	quad.	✗
constrained clustering [50]	d^2	✓	✗	linear	✗
coordinate opt. [45]	d^2	✓	✗	linear	✗
shortest path MGM [24]	d^2	✗	✗	quad.	✗
Our	d	✓	multi	quad.	✓
We contribute to	✓	✓	✓		✓

The problem combining these conditions is known as *multi-graph matching (MGM)* and NP-hard [17, 45] in general. The term *graph* originally stems from interpreting objects as graphs and deriving quadratic costs from differences of respective edge attributes.

MGM is closely related to the multi-index or *multi-dimensional assignment problem* [37] (MDAP) studied in operations research. The MDAP considers only complete matchings and has a different cost structure: Costs are assigned to each d-tuple of keypoints, with one keypoint per object. Respectively, the length of the cost vector grows exponentially with d in general. To mitigate this issue, *decomposable* MDAPs have been proposed, with the d-tuple cost being a sum of individual linear keypoint-to-keypoint costs. The respective subclass of the MDAP corresponds to a special case of the *complete MGM problem without quadratic costs*, introduced as a *complete Multi-LAP* later in Section 2. Despite the strong relation between MGM and the MDAP, the significant progress related to approximation algorithms for the MDAP established in the operations research community for more than 30 years [4, 5, 39], has so far not been utilized in computer vision.

In this work, we bridge this gap by revisiting these algorithms, adapting them to the general, incomplete MGM problem with quadratic costs, and proposing their extended and parallelized versions.

2 Related Work

2.1 Graph Matching

Graph Matching (GM) refers to the well-studied [20] special case of matching *two* objects. Similar to MGM, one distinguishes *complete* and *incomplete* GM. Complete GM is also known as the NP-hard [38] *quadratic assignment problem (QAP)* [29] or, if quadratic costs are zero, the polynomially solvable [28] *linear assignment problem (LAP)* [12]. MGM can be viewed as $d(d-1)/2$ GM problems between all object pairs, coupled via cycle consistency constraints. If we wish to distinguish

between cost orders similar to GM, we refer to (complete *or* incomplete) MGM as Multi-QAP and, respectively, as Multi-LAP if quadratic costs are zero. Unlike the LAP, the Multi-LAP is NP-hard [17, 45]. As a result of this coupled structure, GM solvers are an essential but interchangeable component of many MGM and MDAP methods, including those considered in this work. We utilize the GM solver [22] as a GM subroutine since it shows the best results in the recent GM benchmark [20].

2.2 Multi-Graph Matching

Although MGM has yet to receive the same attention as GM, several solvers have been proposed in recent years. Rather than summarily discussing them, we dissect them and discuss the parts most relevant to our work, *i.e.*, *incompleteness*, *sparsity*, *solution construction*, and *local search*, while bridging the gap between computer vision and operations research, *i.e.*, MGM and the MDAP, see Table 1.

Complete vs. Incomplete MGM. Many approaches [8, 45, 50, 51, 52] only treat complete MGM. Despite popular claims to the contrary, there is no straightforward and efficient way to apply them to the incomplete case. While a polynomial reduction from incomplete to complete GM exists [20], its often-mentioned generalization to MGM, *e.g.*, [50, 51, 52], is prohibitively expensive. Transforming an *incomplete* MGM problem between d objects, each with n keypoints, into a *complete* one results in nd keypoints in *each* of the d objects, see Section 8. In contrast, our approach applies to incomplete problems *directly*.

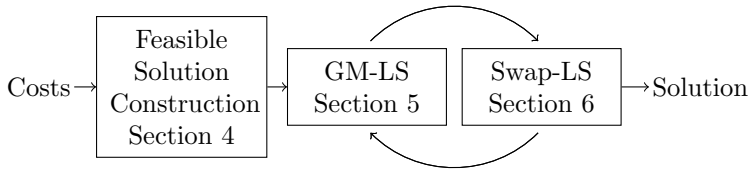
Sparse Problems. We call an MGM problem *sparse* if most keypoint-to-keypoint matchings are *forbidden*, and otherwise *dense*. Since the number of costs necessary to describe a dense problem with n keypoints grows as $O(n^4)$, dense MGM with $n > 100$ becomes practically infeasible. Hence, one must either resort to parametrized costs [10, 53], *e.g.*, Koopmans-Beckmann form [6], or *forbid* high-cost matchings [25, 20]. While the former reduces a problem's expressiveness, the latter increases it by allowing prior knowledge to be incorporated and is, therefore, often preferable. In practice, forbidden matchings are modeled via *infinite* costs that are only implicitly present in the problem description [43, 46, 20], rendering the latter *sparse*. However, many methods, especially the solution construction ones, considered in the following paragraph, do not apply to infinite costs. Even worse, they often fail to reconcile infinite costs with their assumptions, *e.g.*, by heavily relying on spectral methods [7, 2, 36]. In contrast, our method guarantees to return only *allowed*, *finite-cost* matchings for *sparse* problems.

Feasible Solution Construction. Given all constraints, already constructing a feasible, low-cost matching is challenging. In computer vision, three methods prevail: *Synchronization*, *composition*, and *spanning tree* approaches.

Synchronization approaches [14, 42, 15] first independently solve all pairwise GM problems, which induces a *cycle-inconsistent* matching. Afterward, they try to find a *cycle-consistent approximation* by changing a *minimal* number of matchings [2, 7, 31]. This approximation step is often understood as "synchronizing" pairwise solutions and can be formulated as a Multi-LAP or (equivalent for complete problems) *Hamming distance projection* onto the set of consistent matchings [9, 36]. Such synchronization methods often serve as a subroutine for MGM solvers that relax MGM, *e.g.*, the message-passing approach [44] or the convex relaxation approach [8]. However, they are *expensive* and prone to *suboptimal* decisions as they require the solution of $d(d-1)/2$ GM problems and ignore costs during the approximation. Due to the latter, existing synchronization methods also fail to deal with sparse problems because they often introduce *blunders* in choosing forbidden matchings. In contrast, our algorithm can be used as a synchronization method for sparse problems.

Composition methods [24, 49] substitute matchings between two objects by composition via a third, based on the inconsistent pairwise matching. This third object is iteratively chosen w.r.t. a metric combining costs *and* a (pairwise) consistency measure. While, for complete problems, any consistent solution is a fixed point of such updates, convergence to it is not guaranteed. Composition methods, therefore, often rely on "ordinary" synchronization as a postprocessing step.

Spanning Trees of the complete graph with objects as vertices are another way to establish feasible MGM solutions. Predominantly *star-shaped* trees defined by a root or *reference* object are used [50, 52]. After solving all pairwise problems involving the reference object, matchings between any object pair are inferred by composition via the reference object, guaranteeing cycle consistency.



Our MGM method is composed of three parts – a constructive heuristic (Section 4) and two local search heuristics (Sections 5 and 6). In Section 7, we showcase our method by addressing the synchronization of sparse problems. Theoretical insights about reducing incomplete MGM to complete MGM are provided in Section 8.

Figure 2: **Conceptual Diagram of Our Method**

The generalization to arbitrary spanning trees is proposed in [45]. The most prominent drawback is that the construction only uses $O(d)$ out of the $O(d^2)$ costs defining the problem. Precomputing all pairwise matchings and choosing *minimal spanning trees* w.r.t. pairwise costs or consistency is a common [45, 50, 52] but expensive and still limited [5] remedy. In particular, such methods cannot address sparse problems because forbidden matchings are often selected during inference.

In contrast, we adopt the technique [4, 5] proposed within the operations research community aimed at the MDAP. This technique extends a feasible solution consisting of $k \leq d$ objects by one object on each iteration until the solution includes all objects, *i.e.*, $k = d$. Like the spanning tree techniques, our approach only requires the solution of $O(d)$ pairwise problems but considers *all* $O(d^2)$ costs defining the problem.

Local Search. Two complementary local searches, based on *swaps* and solving a *GM* problem, are known for MGM or related problems.

GM Local Search. Cycle-consistent matchings can be improved by re-matching one object to the remaining, already matched $(d-1)$ objects, see Fig. 3. It turns out that this re-matching constitutes a GM problem. Hence, one can iteratively re-match different objects utilizing a GM solver. We refer to this algorithm as *GM local search*, see Section 5 for details. The costs for the underlying GM problems are obtained through an appropriate summation of *all* costs involving the object to be re-matched. This idea was independently proposed for different MGM variants in [45, 50, 51, 52]. In operations research, it is known for the MDAP, along with its generalization of re-matching object *subsets*, since 2004 [5]. We *unify* and *parallelize* it for MGM.

Swap Local Search. Feasible matchings decompose into sets or *cliques* of keypoints that are matched to each other, see Fig. 1. *Swapping* an object's keypoints between these cliques (see Fig. 3) is another, in operations research, well-known way to traverse the search space. Originally proposed in [3] and mostly applied to MDAP variants, it sparked many different heuristics [1, 32, 16, 35], ultimately boiling down to exhaustively searching for the best swaps in a predefined neighborhood. Expanding on [39], we *jointly* consider *all* possible swaps between two cliques but explore this neighborhood efficiently using *quadratic pseudo-boolean optimization* [40, 27].

Local Search Combinations. The complementary nature of the two above-mentioned local searches is revealed by realizing that within a *clique* of matched keypoints, each is uniquely identified by the *object* to which it belongs, see Fig. 3. That is, solutions have two *"dimensions"*. Respectively, these algorithms fix one, either objects or cliques, and recombine over the other, which is why alternating similar variants has succeeded for the MDAP [26, 5]. We do the same for MGM.

2.3 Contribution

Our contribution has theoretical and algorithmic aspects. On the *theoretical* side, we investigate the most common reduction from incomplete to complete MGM, prove its minimality within the class of clique-wise reductions, and, in doing so, expose its prohibitive scaling.

On the *algorithmic* side, we revisit existing primal heuristics for the MDAP and MGM, adopt, combine and extend them to the most general form of MGM. This results in a powerful new MGM algorithm, see Fig. 2.

In particular, we

- adapt, parallelize, and extend the feasible MDAP solution construction method from [4] to incomplete MGM with quadratic costs,

- introduce a parallelized GM local search leading to faster computations, and
- propose a new swap local search that efficiently optimizes over *all* possible keypoint swaps between two given cliques *at once*. Despite the exponentially large search space of this auxiliary problem, an approximate solution is found efficiently by employing quadratic pseudo-boolean optimization.

Our algorithm applies to MGM with linear, quadratic, and even higher-order costs, consequently solving MGM both directly or as a synchronization method. As for synchronization, it guarantees to return only *finite-cost*, feasible multi-matchings and attains better objective values than any other synchronization technique in our experiments, especially for sparse problems. Directly applied, it outperforms the state of the art on all considered datasets regarding both runtime and objective value.

Because our overall method combines and touches on such an extensive part of the MGM and MDAP literature, we, again, clearly distinguish and indicate the contributions of each of its components at the end of the appropriate sections.

Our code is available at `https://github.com/vislearn/multi-matching-optimization`.

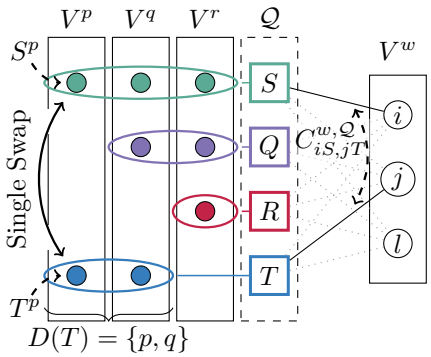
3 Formal Problem Definition

Graph Matching concerns itself with matching two finite keypoint sets V^1 and V^2, further referred to as *vertex sets* or *objects*. It considers the undirected complete bipartite graph $\overline{G} = \left(V := V^1 \cup V^2, \overline{E} := V^1 \times V^2\right)$ with objects V^1 and V^2 as independent sets, where an edge $ij := (i,j) \in \overline{E}$ corresponds to matching vertex $i \in V^1$ to vertex $j \in V^2$. Although discussing undirected graphs, we write the edge set $\overline{E}$ as a Cartesian product $V^1 \times V^2$ to emphasize the two independent sets V^1 and V^2. Due to their independence, we can always identify directed with undirected edges. An *incomplete* matching between object V^1 and V^2 is defined as subset of edges $E \subset \overline{E}$ containing *at most one* incident edge for each vertex. *Complete* matchings, conversely, contain *exactly one* incident edge for each vertex, which demands equal cardinalities of both objects $|V^1| = |V^2|$. The goal of GM is to find minimal matchings w.r.t. given costs $C : \left(V^1 \times V^2\right)^2 \mapsto \mathbb{R}$. In the matrix identification $C(i,s,j,t) = C_{is,jt}$, diagonal entries $C_{is,is}$ describe *linear* costs, and off-diagonal entries *quadratic* costs, see Fig. 1. Linear costs $C_{is,is}$ penalize *vertex-to-vertex* correspondences, *i.e.*, matching vertex $i \in V^1$ to vertex $s \in V^2$, whereby infinite costs $C_{is,is} = \infty$ *forbid* such a matching. Quadratic costs $C_{is,jt}$, in turn, penalize *vertex-pair-to-vertex-pair* correspondences, *i.e.*, matching the pair $(i,j) \in V^1 \times V^1$ to the pair $(s,t) \in V^2 \times V^2$.

Multi-Graph Matching generalizes GM by matching multiple, $d \in \mathbb{N}_{\geq 3}$ objects V^p, $p \in [d] := [1,d] \cap \mathbb{N}$, w.r.t. costs between all pairs $C^{p,q} : \left(V^p \times V^q\right)^2 \mapsto \overline{\mathbb{R}}$, $p \neq q$, $p,q \in [d]$. Instead of a bipartite graph, it considers the undirected complete *d-partite* graph $\overline{G} = \left(V := \cup_{p\in[d]} V^p, \overline{E}\right)$ with objects V^p as independent sets. *Incomplete* multi-matchings are subsets of edges $E \subset \overline{E}$ *s.t.* any vertex is incident to *at most one* edge connecting the same objects. Similar to GM, *complete* multi-matchings require *exactly one* such edge and equal cardinalities of objects.

Cycle Consistency. A multi-matching $E \subset \overline{E}$ is *cycle-consistent* if each *path* in E can be extended within E to a *cycle* with at most one vertex per object. As shown in [44], enforcing this for all 3-cycles is sufficient. That is, a multi-matching $E \subset \overline{E}$ is cycle-consistent iff $ij \in E$ and $jk \in E$ imply $ik \in E$ for all $i,j,k \in V$, see Fig. 1 for an illustration.

Clique Representation. In [47], it was shown that a multi-matching $E \subset \overline{E}$ is cycle-consistent iff the corresponding subgraph $G = (V,E)$ is a *union of cliques*, *i.e.*, there exist partitions $\{Q_l\}_{l\in L}$ of vertices V and $\{E_l\}_{l\in L}$ of edges E indexed via the same finite set L, *s.t.* for each index $l \in L$ the subgraph $G|_{Q_l}$ restricted to part Q_l is a *clique* with edges E_l. Therefore, cycle-consistent multi-matchings are induced by vertex partitions $\mathcal{Q}$ where any part $Q \in \mathcal{Q}$ contains at most one element per object V^p, *i.e.*, $|Q \cap V^p| \leq 1$, see Fig. 1. They translate to multi-matchings by considering elements of the same part Q as matched to each other. We call such vertex partitions *feasible* and denote the set of feasible vertex partitions or *solutions* as $\mathbb{Q}$. The set of feasible partitions over the vertices $V^D := \bigcup_{p\in D} V^p$ of an object *subset* $D \subseteq [d]$ is denoted by $\mathbb{Q}^D$. For simplicity, partitions permit the empty set. Abusing terminology, we refer to parts $Q \in \mathcal{Q}$ of a solution as *cliques*. The object subset actually covered by such a clique $Q \in \mathcal{Q}$ is denoted as $D(Q) \subseteq [d]$, and a clique's vertex belonging to object $p \in D(Q)$ as Q^p, *i.e.*, $\{Q^p\} := Q \cap V^p$, see Fig. 3.



Depicted are three cycle-consistently matched objects V^p, V^q, V^r. Matched vertices are of the same color, horizontally aligned, and decomposed into cliques S, Q, R, T (ellipses), yielding a partial solution $\mathcal{Q}$ (dashed rectangle). E.g. the clique T spans the objects $D(T) = \{p, q\}$, where its vertex T^p belongs to object V^p. The costs $C^{w,\mathcal{Q}}_{iS,jT}$ for matching vertex i and j of object V^w to cliques S and T of the partial solution $\mathcal{Q}$ are also shown. The single swap (solid arrow) between the cliques S and T fixing the object V^p interchanges the vertices S^p and T^p.

Figure 3: **Feasible solution contruction**.

The MGM Objective is to find cycle-consistent multi-matchings minimizing the sum of all costs, *i.e.*,

$$\min_{\mathcal{Q}\in\mathbb{Q}} \left[C(\mathcal{Q}) := \sum_{Q,R\in\mathcal{Q}} \sum_{\substack{p,q\in D(Q)\cap D(R)\\ p<q}} C^{p,q}_{Q^pQ^q,R^pR^q} \right], \tag{1}$$

where we assume $C^{p,q} = C^{q,p}$ and count this cost for each pair of objects $p, q \in [d]$, only once by requiring $p < q$. Note that the formulation in Eq. (1) implicitly assumes zero costs for *not* matching a vertex.

Alternative Formulations. The clique formulation from Eq. (1) is non-standard. Many formulations [44, 54, 15] represent (multi-)matchings $E \subset \overline{E}$ by *partial permutation matrices* X, where $X_{is} = 1$ iff $is \in E$. Others [34, 10, 36] view MGM as a *labeling problem*, where each vertex of the same object must be assigned a different label – commonly called *universe point*. Vertices with the same label are matched to each other, *i.e.*, comprise a clique. We use the clique formulation because it allows the most natural description of our algorithm.

4 Feasible Solution Construction

Basic Algorithm. The basic construction Algorithm 1 obtains feasible solutions by solving a *chain* of GM problems. Given a *random ordering* of objects $[d]$, it iteratively extends a *partial solution*, which, in the k-th iteration, matches the objects $V^1, \ldots, V^k$. This partial solution is a feasible partition $\mathcal{Q} \in \mathbb{Q}^{[k]}$ over the *object subset* $[k] \subseteq [d]$. Its extension $\mathcal{Q}' \in \mathbb{Q}^{[k+1]}$ is obtained through a matching $E \subset V^{k+1} \times \mathcal{Q}$ between the next object V^{k+1} and its cliques $\mathcal{Q}$, see Fig. 3. The matching E is the solution of the GM problem with costs $C^{k+1,\mathcal{Q}}_{iS,jT}, i, j \in V^{k+1}, S, T \in \mathcal{Q}$, stemming from the *summation* over a clique's individual elements

$$C^{k+1,\mathcal{Q}}_{iS,jT} := \sum_{q\in D(S)\cap D(T)} C^{k+1,q}_{iS^q,jT^q} \,. \tag{2}$$

The partial solution is extended by adding matched vertices $i \in V^{k+1}, iS \in E$ to their assigned cliques $\{i\} \cup S \in \mathcal{Q}'$. If a vertex $i \in V^{k+1}$ is unmatched, *i.e.*, $\forall jS \in E : j \neq i$, it is added as a singleton $\{i\} \in \mathcal{Q}'$. If a clique $S \in \mathcal{Q}$ is unmatched, *i.e.*, $\forall iT \in E : S \neq T$, it remains unchanged $S \in \mathcal{Q}'$. We denote the (partial) solution resulting from this *merge* as $\mathcal{Q}' = \mathbf{merge}(V^{k+1}, \mathcal{Q}; E)$.

Algorithm 1 has five notable properties:

1. It is randomized w.r.t. the object ordering.
2. It guarantees cycle-consistency.
3. It is independent of the cost order.
4. It is parameter-free w.r.t. the number of cliques.
5. It scales linearly in the number of objects $d \in \mathbb{N}_{\geq 3}$.

Algorithm 1: Basic Solution Construction.

$\mathcal{Q} \leftarrow \bigcup_{i \in V^1} \{i\}$
for $k \in \{1, \dots, d-1\}$ **do**
  `// compute matching` E `between object` $(k+1)$ `and the already matched` $[k]$
  $E \leftarrow$ (Approximately) solve GM with costs $C^{k+1,\mathcal{Q}}$ from Eq. (2)
  $\mathcal{Q} \leftarrow$ **merge**$(V^{k+1}, \mathcal{Q};\ E)$

Following the GRASP paradigm [18], the randomization can be exploited by running the algorithm multiple times (in parallel) and keeping the best solution. Cycle consistency is guaranteed as the extension procedure preserves the clique structure of the multi-matching E. The quadratic costs from Eq. (2) apply directly to Multi-LAPs and straightforwardly generalize to *higher orders*. Hence, *plug-&-play* of pairwise GM solvers enables Algorithm 1 to address MGM for all cost orders. At any stage, the number of cliques is canonically determined through each (partial) partition. Therefore, contrary to many other methods, *e.g.*, [15, 9], its *implicit* definition through the costs of not matching a vertex suffices. Additionally, Algorithm 1 only requires the solution of $d-1$ GM problems.

Extensions and Parallelization. We propose two extensions of Algorithm 1: *Incremental* and *parallel* construction.

Incremental construction addresses the problem of *error propagation*. Errors during early matchings "propagate" along the chain of pairwise problems, sway later matchings, and worsen the final solution. Because MGM problems' restrictions to object subsets are again MGM problems, we propose to weaken such propagations by using a *better* (but potentially more expensive) *multi-matching solver* for the first $s \in [d]$ objects $[s]$.

Parallel construction improves the linear scaling w.r.t. the number of objects. It generalizes the chain of pairwise problems to *binary trees*, specifically *leaf-labeled, ordered, binary trees* with *leaf label set* $[d]$. Leaf labels associate leaves with objects. Each tree vertex corresponds to a (partial) solution $\mathcal{Q} \in \mathbb{Q}^D$ matching its descendant leaves $D \subseteq [d]$. In analogy to the basic variant, a tree vertex's (partial) solution is obtained by first solving a GM problem that matches the cliques of its children $\mathcal{A} \in \mathbb{Q}^{D_A}$, $\mathcal{B} \in \mathbb{Q}^{D_B}$ and then merging matched cliques. Pairwise costs $C^{\mathcal{A},\mathcal{B}}_{IS,JT}, I, J \in \mathcal{A}, S, T \in \mathcal{B}$, are again obtained by summing costs of a clique's individual elements,

$$C^{\mathcal{A},\mathcal{B}}_{IS,JT} := \sum_{p \in D(I) \cap D(J)} \sum_{q \in D(S) \cap D(T)} C^{p,q}_{I^p S^q, J^p T^q} \,. \tag{3}$$

Ultimately, the root corresponds to a final solution. GM problems of tree vertices at the same level can be solved in *parallel*. Therefore, *balanced* trees yield the best acceleration – in sequence only $O(\log(d))$ instead of $O(d)$ problems need solving. Additionally, most properties of the basic construction transfer to the parallel version: It is *randomized* w.r.t. the leaf order, allows *plug-&-play* of pairwise solvers, guarantees *cycle-consistency*, and is *parameter-free* w.r.t. the number of cliques. Pseudocode and further details are provided in Supplement A.

Related Work. The basic solution construction Algorithm 1 is first mentioned in [4] and later used in [5] for complete Multi-LAPs seen as a special case of the MDAP. Experiments in [5] also demonstrate its superiority over star-shaped spanning tree approaches. Its application to *Multi-QAPs* and its *parallelization* using binary trees are new. Incremental construction conceptually resembles an idea proposed in [45], where, as part of a spanning tree heuristic, a local search is run on the hitherto constructed partial solution. We generalize this idea by employing *arbitrary solvers* and applying it to *Multi-QAPs* instead of Multi-LAPs.

5 GM Local Search

Basic Algorithm. The basic GM local search step comprises *splitting* and *merging* an object V^p from and to a given solution $\mathcal{Q} \in \mathbb{Q}$ as defined by Algorithm 2. An object V^p is split by *subtracting* $Q \setminus V^p$ its vertices from all cliques $Q \in \mathcal{Q}$. Merging it back to the split solution works just as in Section 4 by solving a GM problem with costs from Eq. (2) whose solution dictates the merge. While only accepting improvements, the basic GM local search Algorithm 2 repeats this step along an *object sequence* $(p_k)_{k \in \mathbb{N}}, p_k \in [d]$, until a suitable stopping criterion is met. In practice, we run

Algorithm 2: Basic GM Local Search.

Input: Solution $\mathcal{Q} \in \mathbb{Q}$, Object Sequence $(p_k)_{k \in \mathbb{N}}$
$k \leftarrow 1$
while stopping criterion not met **do**
  $\mathcal{Q}' \leftarrow \{\, Q \setminus V^{p_k} | Q \in \mathcal{Q} \}$ `// split object` k `from the matching` $\mathcal{Q}$
  `// compute matching` E `between object` k `and the rest`
  $E \leftarrow$ (Approximately) solve GM with costs $C^{p_k, \mathcal{Q}'}$ from Eq. (2)
  $\mathcal{Q}' \leftarrow$ **merge**$(V^{p_k}, \mathcal{Q}';\ E)$
  if $C(\mathcal{Q}') < C(\mathcal{Q})$ **then**
    $\mathcal{Q} \leftarrow \mathcal{Q}'$ `// accept if profitable`
  $k \leftarrow k + 1$

Algorithm 2 cyclically along the random object ordering $[d]$ used in Algorithm 1 as long as the solution improves. Like Algorithm 1, the pairwise solver is *blackboxed* and solutions are always *cycle-consistent*.

Extensions and Parallelization. A common extension [5] of this local search is to split a solution $\mathcal{Q} \in \mathbb{Q}$ along a subset of objects $D \subset [d]$. As a result, two partial solutions $\{Q \cap V^D | Q \in \mathcal{Q}\} \in \mathbb{Q}^D$, $\{Q \cap V^{[d] \setminus D} | Q \in \mathcal{Q}\} \in \mathbb{Q}^{[d] \setminus D}$ are merged using Eq. (3). Since these extensions performed worse than the base in preliminary experiments, we did not investigate them further.

Our proposed *parallelized* search step consists of two passes. First, it runs the basic search step for *all* objects $p \in [d]$ in *parallel*, splitting d objects $\mathcal{Q}^p := \{Q \setminus V^p | Q \in \mathcal{Q}\}$, generating d matchings $E^p \subset V^p \times \mathcal{Q}^p$, yielding d proposed solutions, but ultimately leaving the input solution $\mathcal{Q} \in \mathbb{Q}$ untouched. Second, it splits and merges objects according to the collected matchings $E^p, p \in [d]$, in *ascending* order of their proposed solutions' objective value – only accepting profits. The more matchings yield profit despite being "outdated" (because prior matchings changed the solution), the more *acceleration* is generated over the basic search step. Both the basic and parallel search step *sequentially* solve only *one* GM problem. Additionally, the parallelized step always accepts the *most profitable* proposal though this is only a *secondary* source of acceleration, as demonstrated in Section 9.1. Pseudocode and further details can be found in Supplement B.

Related Work. Again, the basic Algorithm 2 was first mentioned in [5] targeting complete Multi-LAPs as special case of the MDAP. In [5], its extension to object subsets is investigated and our preliminary findings are confirmed for the Multi-LAP. In computer vision, it was independently proposed in [45, 50] for Multi-LAPs and in [51, 52] for Multi-QAPs, both only complete. The parallelized extension is novel.

6 Swap Local Search

Single Swap. Given a solution $\mathcal{Q} \in \mathbb{Q}$, *swapping* vertices of the same object between two cliques $Q, R \in \mathcal{Q}$ maintains feasibility. While fixing one object $p \in [d]$, a *single swap* subtracts a vertex covered by either clique from said clique and adds it to the other, yielding two new cliques, Q' and R', see Fig. 3. For example, if both cliques Q and R cover the fixed object $p \in [d]$, the vertices Q^p and R^p are interchanged, *i.e.*, $Q' = (Q \setminus \{Q^p\}) \cup \{R^p\}$ and $R' = (R \setminus \{R^p\}) \cup \{Q^p\}$. If only one clique covers the fixed object, only one vertex is exchanged. Finally, all cliques remain unchanged if none covers the object. After a swap, cliques still cover at most one vertex per object. The new solution $\mathcal{Q}' = (\mathcal{Q} \setminus \{Q, R\}) \cup \{Q', R'\}$ is, thus, also feasible. Furthermore, the objective value change induced by a single swap decomposes into a sum of objective value changes $\delta C^{p,q}_{\text{swap}}(Q, R)$ between the fixed object $p \in [d]$ and all other objects $q \in [d] \setminus \{p\}$, *i.e.*,

$$C(\mathcal{Q}') - C(\mathcal{Q}) = \sum_{q \in [d] \setminus \{p\}} \delta C^{p,q}_{\text{swap}}(Q, R)\,. \tag{4}$$

A detailed formulation is given in Supplement C.

Joint Multi-Swap. The decomposition from Eq. (4) can be leveraged to *jointly* consider *multiple* swaps or *multi-swaps* between two cliques $Q, R \in \mathcal{Q}$. A multi-swap is a set of binary decision variables $x = \{x^p\}_{p \in [d]} \in \{0, 1\}^d$, deciding for every object $p \in [d]$, whether the single swap fixing it should be performed ($x^p = 1$), or not ($x^p = 0$). Performing all single swaps

Algorithm 3: Swap Local Search.

Input: Solution $\mathcal{Q} \in \mathbb{Q}$
while stopping criterion not met **do**
  for $Q, R \in \mathcal{Q}$ **do**
    $x^* \leftarrow$ Approximately solve Eq. (5) for cliques Q, R
    $Q', R' \leftarrow \text{swap}(x^*, Q, R)$ `// swap the vertices according to` x^*
    $\mathcal{Q}' \leftarrow (\mathcal{Q} \setminus \{Q, R\}) \cup \{Q', R'\}$ `// update the clique representation`
    if $C(\mathcal{Q}') < C(\mathcal{Q})$ **then**
      $\mathcal{Q} \leftarrow \mathcal{Q}'$ `// accept if profitable`

specified by a multi-swap $x \in \{0,1\}^d$ yields, again, two new cliques Q' and R', which we denote by $Q', R' = \text{swap}(x; Q, R)$. The optimal multi-swap x^* solves the following quadratic pseudo-boolean optimization problem [11]:

$$x^* \in \arg\min_{x \in \{0,1\}^d} \sum_{p \in [d]} x^p \left(\sum_{q \in [d]} \overline{x^q} \cdot \delta C^{p,q}_{\text{swap}}(Q, R) \right) , \tag{5}$$

where $\overline{x} = (1 - x)$ denotes negation. This problem is NP-hard [40], which is why we solve it using the state of the art [22] approximative solver QPBO-I [40]. Our swap local search Algorithm 3 iterates over all clique pairs, solving the problem from Eq. (5) with QPBO-I and performing the obtained multi-swap if profitable.

Related Work. The single swap operation was originally introduced for the MDAP with three objects [3] and extended in similar variations [32, 39, 26] to multi-swaps. However, these propositions come without a method to solve the joint multi-swap problem efficiently. Our proposal to leverage quadratic pseudo-boolean optimization is new.

7 Use Case: Synchronization and Sparse Problems

Synchronization is a constructive heuristic, that reduces MGM from arbitrary cost orders to the Multi-LAP. It has two stages. First, all $d(d-1)/2$ GM problems (with costs $C^{p,q}$) are *independently* solved, potentially in parallel, yielding matchings $E^{p,q} \subset V^p \times V^q$ between all object pairs $p, q \in [d]$. These matchings induce an, a priori, *inconsistent* multi-matching $E := \bigcup_{p,q \in [d]} E^{p,q}$. Second, a cycle-consistent multi-matching E^* sharing the largest possible number of matchings with E is (approximately) recovered [2, 7, 31], *i.e.*,

$$E^* = \arg\max_F |E \cap F| \qquad \text{s.t. } F \subset \overline{E} \text{ cycle-consistent} \ . \tag{6}$$

The recovering problem (6) can be formulated as a *synchronization Multi-LAP* between all objects V^p with costs

$$M^{p,q}_{is,is} = \begin{cases} -1, & \text{if } is \in E \,, \\ 0, & \text{else} \ . \end{cases} \tag{7}$$

An alternative, which agrees with Eq. (6) for complete problems, is to recover the projection w.r.t. to the *Hamming distance*, *i.e.*, minimizing the symmetric difference $|E \Delta E^*|$ [9, 36].

Synchronization's Achilles' heel is its *ignorance of costs* during its recovery, *i.e.* the second, stage (6). Especially for *sparse* models with a majority of *forbidden* matchings of *infinite* costs $C^{p,q}_{is,is} = \infty$, including just one in the final solution E^* renders it forbidden. Instead, we propose to take *forbidden matchings* into account by solving the above mentioned synchronization Multi-LAP with costs

$$N^{p,q}_{is,is} = \begin{cases} \infty & , \text{ if } C^{p,q}_{is,is} = \infty \,, \\ M^{p,q}_{is,is} & , \text{ else} \,, \end{cases} \tag{8}$$

instead of (7). When used within our algorithms, these costs always lead to *feasible* solutions, containing only *allowed* matchings.

Related Work. Usually, synchronization methods neither discuss sparsity nor apply to infinite costs. The only sparse synchronization method we could find, and this is only through communication with the authors, is a greedy subroutine of [44] not described in the paper. In contrast,

our method, applied to the synchronization Multi-LAP, efficiently and accurately handles the costs from Eq. (8) and, therefore, respects forbidden matchings.

8 Incomplete to Complete Reduction

In the following, we formally state and discuss the *polynomial-time reduction* from incomplete to complete MGM, often mentioned without references or further elaboration in related works, *e.g.*, [50, 51, 52]. To transform an incomplete problem with objects V^p, $p \in [d]$, and costs $C^{p,q}$, $p, q \in [d]$, to a complete one with objects of equal cardinalities, we introduce *dummy* vertices, similar to the respective GM reduction in [20]. More precisely, given that $|V|$ stands for the *total* number of vertices in the original incomplete problem, we add $|V| - |V^p| =: |\Delta^p|$ dummy vertices Δ^p to each object V^p, $p \in [d]$. As a result, we construct a complete MGM problem with objects

$$\widetilde{V}^p = V^p \cup \Delta^p, \quad p \in [d]\,, \tag{9}$$

and costs

$$\widetilde{C}^{p,q}_{is,jt} = \begin{cases} C^{p,q}_{is,jt} & \text{, if } i,s,j,t \in V \\ 0 & \text{, else} \end{cases} . \tag{10}$$

In this way, each vertex $i \in V^p$, $p \in [d]$, of the incomplete problem receives a dummy vertex $\delta \in \Delta^q$ in each object $q \in [d] \backslash \{p\}$ of the complete problem. These dummy vertices are used to turn each clique of the incomplete problem maximal *s.t.* they span all objects of the (complete) problem. In turn, omitting matchings to dummy vertices defines the inverse transformation, which leads us to the following

Theorem 1. *Incomplete MGM is polynomial-time reducible to complete MGM,*

The proof is based on the reduction (9)-(10) and given in Supplement D . However, citing such a transformation, without further reasoning, as a practical way to apply solvers for complete MGM to incomplete problems is *naïve*. The reduction (9)-(10) adds $\sum_{p\in[d]}(|V| - |V^p|) = (d-1)|V|$ dummy vertices, *i.e.*, $d-1$ times the size $|V|$ of the incomplete problem, which renders most existing algorithms impractical by considerably slowing them down.

Yet, the described reduction is *minimal* within the class of *clique-wise* reductions, that is the size of respective complete problems can not be decreased in general:

Definition 1. *A problem class P^A is reducible in polynomial time to the problem class P^B, if (i) there exists a mapping $T_{inst}\colon P^A \to P^B$ and (ii) for any $A \in P^A$ there exists a mapping T_{sol} from the optimal solutions of $T_{inst}(A)$ to the optimal solutions of A, where all mappings are computable in polynomial time.*

Being very general, Definition 1 does not consider several practical issues: First, an optimal solution is rarely known a priori. Therefore, the mapping T_{sol} is usually defined for *all* solutions that can potentially become optimal. In case of the MGM problem these are all feasible multi-matchings, as for each of them costs exists that turn them optimal. Second, if approximate algorithms are used to solve the reduced problem, then the optimum-to-optimum mapping, as required in Definition 1, is insufficient. Instead, the mapping T_{sol} must be *monotone* w.r.t. the objective values of the mapped feasible solutions: Feasible solutions with lower objective values in $T_{inst}(A)$ correspond to feasible solutions with lower objective values in A. These properties are summarized in the following

Definition 2. *We say that there is a* robust *polynomial time* reduction *of a problem class P^A to the problem class P^B, if (i) there exists a mapping $T_{inst}\colon P^A \to P^B$ and (ii) for any $A \in P^A$ there exists a* monotone *mapping T_{sol} from the feasible solutions of $T_{inst}(A)$ to the feasible solutions of A, where all mappings are computable in polynomial time.*

Let now P^A and P^B correspond respectively to the problem classes of incomplete and complete MGM. Any feasible solution of an arbitrary MGM problem A' is given by its vertex partition $\mathcal{Q}^{A'} \in \mathbb{Q}^{A'}$. In the following, we define a subset of robust reductions, where each clique in the partition $\mathcal{Q}^A$, $A \in P^A$, has its counterpart in the *respective* partition $\mathcal{Q}^B$, $B = T_{inst}(A)$:

Definition 3. *Assume $B = T_{inst}(A)$ and $\mathcal{Q}^A := T_{sol}(\mathcal{Q}^B)$. We call the robust reduction $P^A \to P^B$* clique-wise*, if the mapping T_{sol} is induced by a clique-wise mapping $T_{clique} : \mathcal{Q}^B \to \mathcal{Q}^A$* s.t. $T_{clique}\big|_{\mathcal{Q}^B \setminus T^{-1}_{clique}(\emptyset)}$ *is injective, which directly implies $|\mathcal{Q}^A| \leq |\mathcal{Q}^B|$.*

As it follows from the proof of Theorem 1, the reduction (9)-(10) is clique-wise. We are not aware of any non-clique-wise reductions, although cannot exclude their existence. Definition 3 leads to the following important

Theorem 2. *Let $B \coloneqq T_{inst}(A) \in P^B$ be a complete MGM problem instance that is a clique-wise reduction of an incomplete MGM problem instance $A \in P^A$. Let A have $|V|$ vertices in total. Than* each object *in B has at least $|V|$ vertices.*

Proof. Consider the feasible multi-matching in A which leaves all vertices unmatched. This has $|\mathcal{Q}^A| = |V|$ cliques representing individual vertices. On the other side, the number of (maximal) cliques in any (complete) MGM solution in B is equal to the number of vertices in each of its objects. Hence, the latter cannot be less than $|V|$ due to Definition 3. □

9 Experimental Validation

Our experimental evaluation is split into an *ablation study* and *a comparison to other methods*. Whereas the ablation study compares different variants of our algorithms, during the comparison to others, we stick to the basic sequential variants of the algorithms for fair runtime comparisons. All experiments were run with a single core on an AMD Milan EPYC 7513.

In this regard, the best-performing sequential algorithm, referred to as our method (*Our*), constructs solutions using the basic construction Algorithm 1 (*Our-C*) and improves them using a local search (*Our-LS*) that alternates between the basic GM local search Algorithm 2 (*GM-LS*) and the swap local search Algorithm 3 (*SWAP-LS*), while solving GM problems using the state-of-the-art GM solver [22, 23]. The latter is run with the following otherwise default parameters: Batch size 10, 10 greedy generations per batch, number of batches 10. Additionally, we start with a *fixed* randomization seed 42, which turns the otherwise randomized solver [22, 23] into a deterministic one. This is needed to separate the randomization effects of *Our-C* from those of the underlying GM solver.

In order to bridge both sections, *Our* is also used as a baseline during the ablation study.

Contrary to most existing works, we only compare attained *objective values* or *costs* instead of *ground-truth-based accuracies* since none of the considered algorithms explicitly optimize the latter or has access to the ground truth. Moreover, we often found the available ground truth to be highly sub-optimal in terms of costs, which would notably distort the respective comparison. Therefore, we leave accuracy comparisons to the works addressing modeling and learning questions.

Datasets. We evaluate methods on the established *synthetic*, CMU *hotel* and *house*, and *worms* datasets, described in [43] and accessible through the accompanying archive. We summarize the characteristics of these datasets in Table 2.

Additionally, to showcase our method, we extended the *worms* dataset by a new, large problem instance, *worms-29*, by combining all 29 worm-based objects available in the *worms* dataset. In this instance, we reduced the size of each pairwise problem by forbidding multiple high-cost vertex-to-vertex matchings, making the problem even sparser: Each vertex is allowed to match 14 instead of 23 other vertices on average. All other characteristics, including costs and number of vertices per object, are the same as in *worms*.

9.1 Ablation Study: Construction and Local Search Variants

We compare the basic *sequential (seq-c)*, *parallel (par-c)*, and *incremental (inc-c)* solution construction methods from Section 4 in all possible combinations with the *sequential (seq-ls)* and *parallel (par-ls)* GM local search from Section 5. For lack of a better solver, *inc-c* uses *Our* for the first 5 objects. To showcase our parallelization, we additionally consider a straightforward "best-improvement" parallelization of the GM local search (*best-ls*): Although it performs all GM-LS steps in parallel, it accepts only the most profitable one. In this respect it differs from *par-ls*, which aims to accept *all possible* profitable changes. All experiments were run on *worms-10*, where the depicted results are the average over the first 10 instances. Parallel methods were granted 10 processing cores and all experiments were run on an AMD Milan EPYC 7513.

Table 2: Datasets used for experimental validation. **#obj.** - number of objects per instance; **#vert./obj.** - number of vertices per object; **#inst.** - number of instances in the dataset; **incompl.** and **sparse** - incomplete/complete and sparse/dense problem instances resp. All datasets but *worms-** have four subsets, with 4, 8, 12 and 16 objects respectively. The *worms* datasets has 8 subsets with 50 instances per subset. We test our method on the first 10 instances in each subset. The column **real** shows whether the dataset represents a real-life problem. Although *hotel* and *house* are based on real images, their practical value is limited, as they represent a solid body matching problem solvable with better scalable *structure-from-motion* methods.

dataset	#obj.	#vert./obj.	#inst.	incompl.	sparse	real
synthetic						
├ complete	4, 8, 12, 16	10	4x10	✗	✗	✗
├ density	4, 8, 12, 16	10	4x10	✗	✗	✗
├ deform	4, 8, 12, 16	10	4x10	✗	✗	✗
└ outlier	4, 8, 12, 16	10	4x10	✓	✗	✗
hotel	4, 8, 12, 16	10	4x10	✓	✗	✓/✗
house	4, 8, 12, 16	10	4x10	✓	✗	✓/✗
worms	3,4,5,6,7,8,9,10	500-600	8x10 (8x50)	✓	✓	✓
worms-29	29	500-600	1	✓	✓	✓

Objective Value. To assess the influence of randomization, we run each algorithm 100 times using different object orderings. Fig. 4a shows the resulting cost distributions obtained through Gaussian kernel density estimation. Although the cost distributions significantly differ for different variants of the construction method, this difference vanishes after the local search. In light of this, we disregard *inc-c* as the most expensive method in further experiments. Moreover, *par-c*'s improved scaling can be fully utilized because local searches compensate the cost difference to *seq-c*.

Convergence Speed. Fig. 4b depicts costs over runtimes. While all methods converge to negligibly different values, *par-ls* converges the fastest. Notably, it significantly outperforms the best-improvement parallelization method *best-ls*.

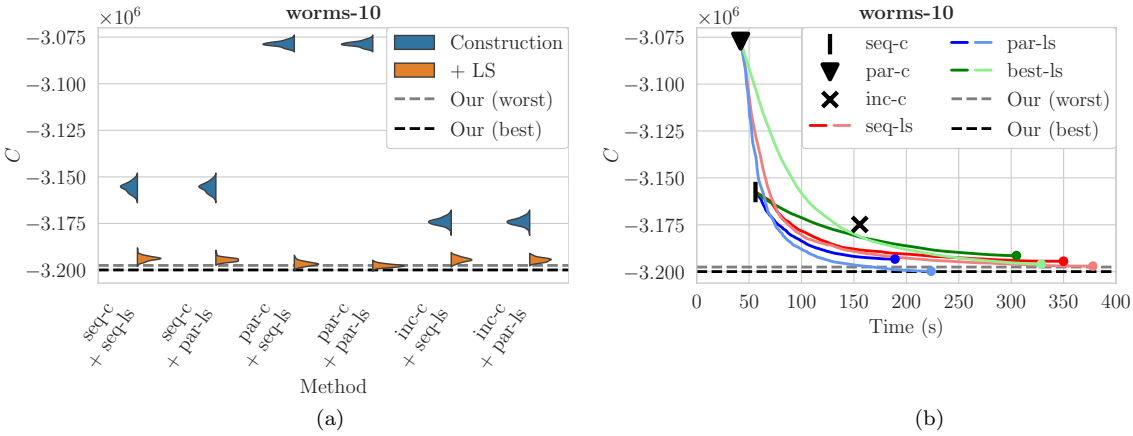


Figure 4: Comparison of the construction and GM local search algorithm variants. Results averaged over 10 instances of *worms-10*. For reference, the best and the worst result of *Our* over 10 runs is depicted as a baseline. ((a)) Cost distributions over 100 runs. Result after construction with either *seq-c*, *par-c* or *inc-c* (initial) and after improving with either *seq-ls* or *par-ls* (+ LS). ((b)) Convergence of *seq-ls*, *par-ls* and *best-ls* variants. Two plots are shown for each algorithm, for *seq-c* and *par-c* results as a starting point respectively.

9.2 Comparison to Other Methods

Algorithms.

Direct MGM Methods. We compare *Our* to the state of the art MGM methods *MP-T* [44] and *DS** [8]. While the latter only applies to complete, dense MGM, the former can solve incomplete and sparse MGM.

Synchronization Algorithms. To compare synchronization algorithms, we first solve all $d(d-1)/2$ pairwise GM problems with [22] and then apply different synchronization techniques to obtain a cycle-consistent solution. A variant of our algorithm further referred to as *Our (Syn)* addresses the synchronization Multi-LAP, equivalent to the problem (6), with the costs (7) or (8). It works just as *Our*, except that it solves all intermediate GM subproblems exactly instead of approximately, as the latter are LAPs and thus efficiently solvable.

We compare *Our (Syn)* to the projected power iteration (*PPI*) [14], Sparse Stiefel Synchronization (*Stiefel*) [7], and the spectral approach [36] extended by the Successive Block Rotation Algorithm [9] to yield incomplete matchings (*Spectral*).

Additionally, we repurpose *MP-T* as a synchronization method *MP-T (Syn)* and apply it to the Multi-LAP (6) with the sparsity-accounting costs (8). The latter is possible as *MP-T* applies to sparse Multi-QAPs and, thus, Multi-LAPs.

Spanning-Tree Based Heuristics. Furthermore, we compare to the constructive heuristics based on arbitrary [45] (*MST*) and star-shaped [52] spanning trees (*MST-Star*). Just like synchronization techniques, these methods require an inconsistent pairwise solution as initialization. We use the same inconsistent solution for the initialization of the spanning tree and synchronization methods.

Implementation Details. While *Our* and *MP-T* are available in C++, *DS**, *Stiefel*, *PPI*, and *Spectral* are Matlab implementations. We use the original author's code for all but *PPI*, which we re-implemented ourselves.

Results. For clarity, we provide here a comparison on selected, representative datasets, whereas the complete list of attained results by all algorithms for all datasets (a total of 321 problem instances in 33 tables) can be found in Supplement E.

Table 3: Results of synchronization methods for *worms-10*, averaged over 10 instances, evaluated on the two objectives for synchronization problems, *Multi-LAP objective* (6) (M-LAP obj.), *Hamming distance* (Hamm. dist), see Section 7. We further list the number of forbidden matchings, # forb., of the final solution and it's *MGM objective* value w.r.t. Eq. (1) (MGM Obj.). Using the solution as a starting point, *MGM Obj. (+LS)* is the objective value reached after applying *Our-LS*.
Since neither PPI, nor Spectral or Stiefel are able to solve the sparse synchronization problem with costs (8), we resort to *soft constraints* in the respective experiment by replacing infinite $N^{p,q}_{is,is}$ values in (8) with a finite value $\alpha > 0$. But even then, only *PPI* was able to return sensible results. As a baseline, the results of *Our* and *MP-T* are listed in the last three rows. For *Our*, we give the best and worst result over 10 runs, to account for the impact of randomization on the algorithm.

Inf. Value	Method	M-LAP obj. ↓	Hamm. dist. ↓	# forb. ↓	MGM Obj. ↓	MGM Obj. (+LS) ↓
$\alpha = 0$ (Eq. (7))	PPI [14]	-42951.10	153.05	1428.20	inf	-3203810.76
	Spectral [36, 9]	-42879.70	152.64	1188.00	inf	-3204288.13
	Stiefel [7]	-43097.10	150.94	1370.80	inf	-3204015.00
	Our (Syn)	**-43374.5**	**147.49**	**279.40**	inf	**-3205224.70**
$\alpha = 1$	PPI [14]	-42819.50	154.64	542.60	inf	-3204011.99
$\alpha = 10$	PPI [14]	-40844.10	167.17	613.40	inf	-3203058.44
$\alpha = 50$	PPI [14]	-39214.10	176.56	779.60	inf	-3201924.17
$\alpha = \infty$ (Eq. (8))	MST [45]	-35443.30	194.10	807.60	inf	-3200176.14
	MST-Star [52]	-38897.70	177.54	386.80	inf	-3201215.12
	MP-T (Syn)	-42699.90	149.64	**0.0**	-3116929.76	-3205701.80
	Our (Syn)	**-43035.30**	**147.06**	**0.0**	**-3132576.73**	**-3206926.82**
baseline	MP-T [44]	-	-	**0.0**	-2393020.72	-3197584.63
	Our (worst)	-	-	**0.0**	-3148317.88	-3196554.82
	Our (best)	-	-	**0.0**	**-3155013.39**	**-3200248.57**

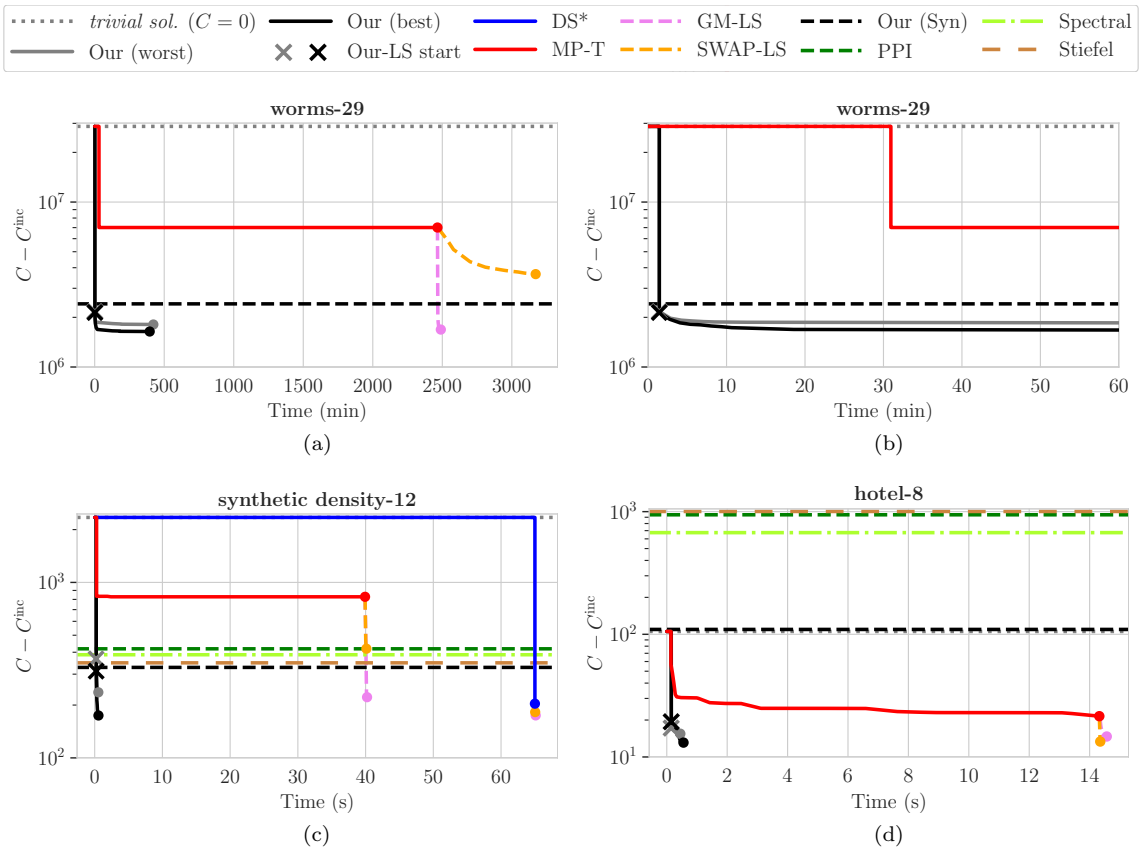


Figure 5: Objective over time comparison. Objective plotted in log-scale and offset by the *inconsistent* solution's objective C^{inc} obtained from independently solving $d(d-1)/2$ pairwise GM problems. The value C^{inc} approximates, since [22] is approximative, a lower bound to Eq. (1) provided by the MGM relaxation that ignores cycle consistency.
Results are averaged over all instances of the dataset. For *Our*, the best and worst result over 10 runs is given. (b) The first 60 minutes of Fig. 5a, illustrating the considerable difference in construction time between *Our* and *MP-T* for large problems. (c) and (d) provide averaged results over all 12 object *synthetic density* and all 8 object *hotel* datasets respectively. (a), (b) compare only methods able to find allowed solutions of sparse problems. *DS** results are shown for (c) only, as the other datasets contain incomplete problems. Note, that even 10 sequential runs of *Our* to attain *Our(best)*'s results would still render the fastest algorithm.

Synchronization Results. The comparison between all synchronization methods and tree-based heuristics are summarized in Table 3. On every metric, *Our (Syn)* achieves the best results. Synchronization techniques *PPI*, *Spectral*, *Stiefel* could not avoid forbidden matchings in none of the considered experiments. Similarly, the spanning-tree based algorithms *MST* and *MST-Star* not only return forbidden matchings, but also underperform in terms of the synchronization objectives *M-LAP Obj.* and *Hamm. dist.*

Application of *Our-LS* on top of any other method always leads to *allowed* multi-matchings. As already observed in Fig. 5, corresponding objective values negligibly (max. 0.1%) differ from each other, depending on the starting point.

Objective Over Time Comparison. Fig. 5 depicts the objective over time achieved by different methods. Because of incomparable implementations (C++, Python, Matlab), we only consider the final objective value for synchronization methods and not their runtime. Overall, *Our* method consistently outperforms all competitors in terms of attained objective value and runtime (where applicable). In turn, *Our (Syn)* is the best synchronization method.

In particular, in Fig. 5a, *Our* achieves a better solution much faster than the direct competitor *MP-T*, improving construction time from half an hour to just two minutes (see Fig. 5b). While *MP-T* fails to improve upon its initial solution significantly, our local search subroutines *GM-LS* and *SWAP-LS* quickly improve the result but converge slowly. In our current implementation,

SWAP-LS is the main bottleneck due to its quadratic scaling in the number of vertices, which is particularly large for the *worms* dataset.

As mentioned in Section 2, synchronization methods may introduce blunders that substantially impact the resulting cost. This can be seen in Fig. 5d, where all synchronization methods return a solution that is worse than the trivial one, which would leave all vertices unmatched.

Fig. 5c includes a comparison to *DS**, which yields high-quality results but is substantially slower than the other methods, partially due to its Matlab implementation. Because it is a solver for complete problems, it is not considered for the *hotel*, *house*, or *worms* datasets, as the incomplete to complete problem transformation would increase the runtime even further, see Section 8.

All plots in Fig. 5 additionally show the application of the *GM-LS* and *SWAP-LS* local search methods to the results of the other methods. Although the resulting objective values are comparable to *Our*, optimization takes significantly longer.

Discussion and Limitations. Our method shows the best results for both major multi-graph matching use cases: As a direct MGM solver *and* as a synchronization algorithm in sequential *and* parallel settings. Our constructive heuristic is fast and efficient, and the local search procedures attain nearly identical results from all considered starting points. We attribute this to their ability to efficiently explore the exponentially large neighborhood of a solution, which itself largely rests on the efficiency of the underlying graph matching solver utilized in our method [22, 23].

However, we believe that even more accurate practical algorithms are possible: For example, by combining the consistency-enforcing dual updates from the MP-T algorithm [44] with primal heuristics from [22] and the current work.

The modern approach to MGM includes learning costs with neural networks. Although a detailed discussion of such methods is beyond the scope of this work, our solver can be efficiently utilized within them, due to its low runtime.

10 Acknowledgements

This work was supported by the DFG grants number 498181230 and 539435352. Authors further acknowledge facilities for high throughput calculations bwHPC of the state of Baden-Württemberg (DFG grant INST 35/1597-1 FUGG) as well as Center for Information Services and High Performance Computing (ZIH) at TU Dresden.

References

[1] Aiex, R.M., Resende, M.G., Pardalos, P.M., Toraldo, G.: Grasp with path relinking for three-index assignment. INFORMS Journal on Computing **17**(2), 224–247 (2005)

[2] Arrigoni, F., Maset, E., Fusiello, A.: Synchronization in the symmetric inverse semigroup. In: Image Analysis and Processing-ICIAP 2017: 19th International Conference, Catania, Italy, September 11-15, 2017, Proceedings, Part II 19. pp. 70–81. Springer (2017)

[3] Balas, E., Saltzman, M.J.: An algorithm for the three-index assignment problem. Operations Research **39**(1), 150–161 (1991)

[4] Bandelt, H.J., Crama, Y., Spieksma, F.C.: Approximation algorithms for multi-dimensional assignment problems with decomposable costs. Discrete Applied Mathematics **49**(1-3), 25–50 (1994)

[5] Bandelt, H.J., Maas, A., Spieksma, F.C.: Local search heuristics for multi-index assignment problems with decomposable costs. Journal of the Operational Research Society **55**(7), 694–704 (2004)

[6] Beckman, M., Koopmans, T.C.: Assignment problems and the location of economic activities. Econometrica (1957)

[7] Bernard, F., Cremers, D., Thunberg, J.: Sparse quadratic optimisation over the stiefel manifold with application to permutation synchronisation. Advances in Neural Information Processing Systems **34**, 25256–25266 (2021)

[8] Bernard, F., Theobalt, C., Moeller, M.: DS*: Tighter lifting-free convex relaxations for quadratic matching problems. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2018)

[9] Bernard, F., Thunberg, J., Goncalves, J., Theobalt, C.: Synchronisation of partial multi-matchings via non-negative factorisations. Pattern Recognition **92**, 146–155 (2019)

[10] Bernard, F., Thunberg, J., Swoboda, P., Theobalt, C.: Hippi: Higher-order projected power iterations for scalable multi-matching. In: Proceedings of the IEEE International Conference on Computer Vision. pp. 10284–10293 (2019)

[11] Boros, E., Hammer, P.: Pseudo-boolean optimization. Discrete Applied Mathematics (2002)

[12] Burkard, R., Dell'Amico, M., Martello, S.: Assignment Problems. SIAM (2009)

[13] Chen, J., Han, G., Xu, A., Cai, H.: Identification of multidimensional regulatory modules through multi-graph matching with network constraints. IEEE Transactions on Biomedical Engineering **67**(4), 987–998 (2019)

[14] Chen, Y., Candès, E.J.: The projected power method: An efficient algorithm for joint alignment from pairwise differences. Communications on Pure and Applied Mathematics **71**(8), 1648–1714 (2018)

[15] Chen, Y., Guibas, L.J., Huang, Q.X.: Near-optimal joint object matching via convex relaxation. arXiv preprint arXiv:1402.1473 (2014)

[16] Clemons, W.K., Grundel, D.A., Jeffcoat, D.E.: Applying simulated annealing to the multidimensional assignment problem. In: Theory and algorithms for cooperative systems, pp. 45–61. World Scientific (2004)

[17] Crama, Y., Spieksma, F.C.: Approximation algorithms for three-dimensional assignment problems with triangle inequalities. European Journal of Operational Research **60**(3), 273–279 (1992)

[18] Feo, T.A., Resende, M.G.: Greedy randomized adaptive search procedures. Journal of global optimization **6**, 109–133 (1995)

[19] Gao, M., Lahner, Z., Thunberg, J., Cremers, D., Bernard, F.: Isometric multi-shape matching. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 14183–14193 (2021)

[20] Haller, S., Feineis, L., Hutschenreiter, L., Bernard, F., Rother, C., Kainmüller, D., Swoboda, P., Savchynskyy, B.: A comparative study of graph matching algorithms in computer vision. In: European Conference on Computer Vision. pp. 636–653. Springer (2022)

[21] Heimann, T., Meinzer, H.P.: Statistical shape models for 3D medical image segmentation: A review. Medical Image Analysis (2009)

[22] Hutschenreiter, L., Haller, S., Feineis, L., Rother, C., Kainmüller, D., Savchynskyy, B.: Fusion moves for graph matching. In: Proceedings of the IEEE International Conference on Computer Vision (2021)

[23] Hutschenreiter, L., Haller, S., Feineis, L., Rother, C., Kainmüller, D., Savchynskyy, B.: Fusion moves for graph matching website (2021), `https://vislearn.github.io/libmpopt/iccv2021/`

[24] Jiang, Z., Wang, T., Yan, J.: Unifying offline and online multi-graph matching via finding shortest paths on supergraph. IEEE transactions on pattern analysis and machine intelligence **43**(10), 3648–3663 (2020)

[25] Kainmueller, D., Jug, F., Rother, C., Myers, G.: Active graph matching for automatic joint segmentation and annotation of C. elegans. In: Proceedings of the International Conference on Medical Image Computing and Computer Assisted Intervention (2014)

[26] Karapetyan, D., Gutin, G.: Local search heuristics for the multidimensional assignment problem. Journal of Heuristics **17**, 201–249 (2011)

[27] Kolmogorov, V., Rother, C.: Minimizing nonsubmodular functions with graph cuts – a review. IEEE Transactions on Pattern Analysis and Machine Intelligence (2007)

[28] Kuhn, H.W.: The hungarian method for the assignment problem. Naval Research Logistics Quarterly (1955)

[29] Lawler, E.L.: The quadratic assignment problem. Management Science (1963)

[30] Liu, C., Lou, C., Wang, R., Xi, A.Y., Shen, L., Yan, J.: Deep neural network fusion via graph matching with applications to model ensemble and federated learning. In: International Conference on Machine Learning. pp. 13857–13869. PMLR (2022)

[31] Maset, E., Arrigoni, F., Fusiello, A.: Practical and efficient multi-view matching. In: Proceedings of the IEEE International Conference on Computer Vision. pp. 4568–4576 (2017)

[32] Murphey, R.A., Pardalos, P.M., Pitsoulis, L.S.: A greedy randomized adaptive search procedure for the multitarget multisensor tracking problem. Network design: Connectivity and facilities location **40**, 277–302 (1997)

[33] Nie, W., Liu, A., Hao, Y., Su, Y.: View-based 3d model retrieval via multi-graph matching. Neural Processing Letters **48**, 1395–1404 (2018)

[34] Nurlanov, Z., Schmidt, F.R., Bernard, F.: Universe points representation learning for partial multi-graph matching. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 37, pp. 1984–1992 (2023)

[35] Oliveira, C.A., Pardalos, P.M.: Randomized parallel algorithms for the multidimensional assignment problem. Applied Numerical Mathematics **49**(1), 117–133 (2004)

[36] Pachauri, D., Kondor, R., Singh, V.: Solving the multi-way matching problem by permutation synchronization. In: Advances in Neural Information Processing Systems. pp. 1860–1868. Citeseer (2013)

[37] Pardalos, P.M., Pitsoulis, L.S.: Quadratic and multidimensional assignment problems. Nonlinear optimization and related topics pp. 235–256 (2000)

[38] Pardalos, P.M., Rendl, F., Wolkowicz, H.: The quadratic assignment problem - a survey and recent developments. Quadratic Assignment and Related Problems (1993)

[39] Robertson, A.J.: A set of greedy randomized adaptive local search procedure (grasp) implementations for the multidimensional assignment problem. Computational Optimization and Applications **19**(2), 145–164 (2001)

[40] Rother, C., Kolmogorov, V., Lempitsky, V.S., Szummer, M.: Optimizing binary MRFs via extended roof duality. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2007)

[41] Sahillioğlu, Y.: Recent advances in shape correspondence. The Visual Computer **36**(8), 1705–1721 (2020)

[42] Shen, Y., Huang, Q., Srebro, N., Sanghavi, S.: Normalized spectral map synchronization. Advances in neural information processing systems **29** (2016)

[43] Swoboda, P., Hornakova, A., Roetzer, P., Savchynskyy, B., Abbas, A.: Structured prediction problem archive. arXiv preprint arXiv:2202.03574 (2022)

[44] Swoboda, P., Kainmüller, D., Mokarian, A., Theobalt, C., Bernard, F.: A convex relaxation for multi-graph matching. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 11156–11165 (2019)

[45] Tang, D., Jebara, T.: Initialization and coordinate optimization for multi-way matching. In: Artificial Intelligence and Statistics. pp. 1385–1393. PMLR (2017)

[46] Torresani, L., Kolmogorov, V., Rother, C.: A dual decomposition approach to feature correspondence. IEEE Transactions on Pattern Analysis and Machine Intelligence (2013)

[47] Tron, R., Zhou, X., Esteves, C., Daniilidis, K.: Fast multi-image matching via density-based clustering. In: Proceedings of the IEEE International Conference on Computer Vision. pp. 4057–4066 (2017)

[48] Yadav, R., Dupé, F.X., Takerkart, S., Auzias, G.: Population-wise labeling of sulcal graphs using multi-graph matching. Plos one **18**(11), e0293886 (2023)

[49] Yan, J., Cho, M., Zha, H., Yang, X., Chu, S.M.: Multi-graph matching via affinity optimization with graduated consistency regularization. IEEE transactions on pattern analysis and machine intelligence **38**(6), 1228–1242 (2015)

[50] Yan, J., Ren, Z., Zha, H., Chu, S.: A constrained clustering based approach for matching a collection of feature sets. In: 2016 23rd International Conference on Pattern Recognition (ICPR). pp. 3832–3837. IEEE (2016)

[51] Yan, J., Tian, Y., Zha, H., Yang, X., Zhang, Y., Chu, S.M.: Joint optimization for consistent multiple graph matching. In: Proceedings of the IEEE international conference on computer vision. pp. 1649–1656 (2013)

[52] Yan, J., Wang, J., Zha, H., Yang, X., Chu, S.: Consistency-driven alternating optimization for multigraph matching: A unified approach. IEEE Transactions on Image Processing **24**(3), 994–1009 (2015)

[53] Yan, J., Xu, H., Zha, H., Yang, X., Liu, H., Chu, S.: A matrix decomposition perspective to multiple graph matching. In: Proceedings of the IEEE international conference on computer vision. pp. 199–207 (2015)

[54] Zhou, X., Zhu, M., Daniilidis, K.: Multi-image matching via fast alternating minimization. In: Proceedings of the IEEE international conference on computer vision. pp. 4032–4040 (2015)

Unlocking the Potential of Operations Research for Multi-Graph Matching

Supplementary Material

Table of Contents

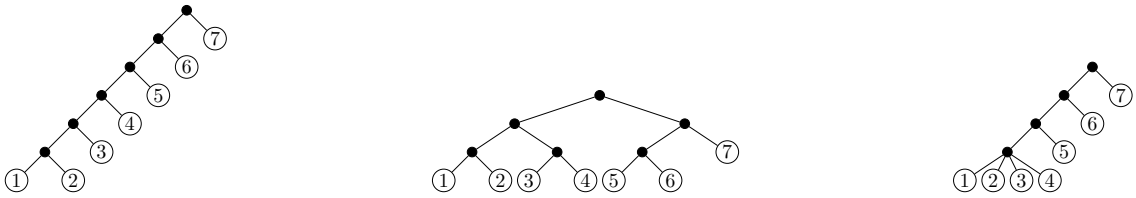
A Feasible Solution Construction: Details **2**

B GM Local Search: Details **3**

C Swap Energy Change Formula **3**

D Proof for Polynomial Reduction from Incomplete to Complete MGM **4**

E Detailed Objective Values per Instance **7**



(a) Basic (b) Parallel (c) Incremental

Figure A.1: **Construction Trees.** All three of our proposed construction variants, *basic* (a), *parallel* (b), and *incremental* (c) construction, can be described via construction trees. Shown are example trees for each variant and $d = 7$ objects.

Algorithm A.1: Parallel Solution Construction.

Input: Binary Construction Tree `T`
for $l \in \{$`T.height`, `T.height`-1, ..., 1$\}$ **do**
  for `vertex` in `T.level`$[l]$ **do in parallel**
    $\mathcal{A} \leftarrow$ `vertex.left.solution`
    $\mathcal{B} \leftarrow$ `vertex.right.solution`
    $E \leftarrow$ Solve GM with costs $C^{\mathcal{A},\mathcal{B}}$ from Eq. (3)
    `vertex.solution` $\leftarrow$ **merge**$(\mathcal{A}, \mathcal{B}; E)$

A Feasible Solution Construction: Details

Merge Operations

The merge operation between two partial solutions $\mathcal{A} \in \mathbb{Q}^{D_A}$ and $\mathcal{B} \in \mathbb{Q}^{D_B}$ of disjoint $D_A \cap D_B = \emptyset$ object subsets $D_A, D_B \subset [d]$ specified via the matching $E \in \mathcal{A} \times \mathcal{B}$ is formally defined as

$$\begin{aligned} \mathbf{merge}(\mathcal{A}, \mathcal{B}; E) \quad &:= \quad \{A \cup B \,|\, AB \in E\} \\ &\cup \quad \{A \,|\, \forall B \in \mathcal{B} : AB \notin E\} \\ &\cup \quad \{B \,|\, \forall A \in \mathcal{A} : AB \notin E\}\,. \end{aligned} \tag{11}$$

The merge between an object V^p and a partial solution $\mathcal{Q} \in \mathbb{Q}^D$, $p \notin D \subset [d]$, specified by the matching $E \subset V^p \times \mathcal{Q}$ is a special case

$$\mathbf{merge}(V^p, \mathcal{Q}; E) := \mathbf{merge}(\mathcal{V}^p, \mathcal{Q}; E')\,, \tag{12}$$

where $\mathcal{V}^p = \{\{i\} \,|\, i \in V^p\}$ and $E' = \{\{i\} \cup Q \,|\, iQ \in E\}$.

Extensions and Parallelization

At the heart of both extensions lie *leaf-labeled*, *ordered* trees with *label set* $[d]$, further referred to as *construction trees*, see Fig. A.1. As described in Section 4, each `vertex` of a construction tree can be associated with a partial solution (`vertex.solution`) for the objects labeled by its descendant leaves. While leaves are always associated with the trivial solution $\mathcal{V}^p = \{\{i\} \,|\, i \in V^p\}$ of the object they label V^p, $p \in [d]$, solutions are generally the result of (matching and) merging the solutions of their children.

Our *parallel construction* limits itself to *binary* construction trees (see Fig. A.1), which is why it can be implemented with only a GM solver, see Algorithm A.1 for pseudocode. The basic construction Algorithm 1 can be identified as the special case of binary construction trees with height $d-1$, see Fig. A.1. It allows for no parallelization as only matchings for solutions of vertices at the same level of a construction tree can be computed in parallel.

Arbitrary construction trees are the idea behind *incremental construction*, where solutions for vertices with more than two children require an *MGM solver*, see Fig. A.1. However, in Section 4 and Section 9, we limit ourselves to trees with only one such vertex at level 2, see Fig. A.1. The idea is to use a (potentially) better MGM solver for the first few objects to warm-start the construction with a better partial solution, which we deemed to be the most promising use case.

B GM Local Search: Details

Pseudocode for the *parallel* GM local search can be found in Algorithm B.1.

Algorithm B.1: Parallel GM Local Search.

Input: Solution $\mathcal{Q} \in \mathbb{Q}$
while stopping criterion not met **do**
  for $p \in [d]$ **do in parallel**
    $\mathcal{Q}' \leftarrow \{Q \setminus V^p \,|\, Q \in \mathcal{Q}\}$
    $E^p \leftarrow$ Solve GM with costs $C^{p,\mathcal{Q}'}$ from Eq. (2)
    $c^p \leftarrow C(\textbf{merge}(V^p, \mathcal{Q}'; E^p))$
  for $p \in [d]$ in ascending order of c^p **do**
    $\mathcal{Q}' \leftarrow \{Q \setminus V^p \,|\, Q \in \mathcal{Q}\}$
    $\mathcal{Q}'' \leftarrow \textbf{merge}(V^p, \mathcal{Q}'; E^p)$
    if $C(\mathcal{Q}'') < C(\mathcal{Q})$ **then**
      $\mathcal{Q} \leftarrow \mathcal{Q}''$

C Swap Energy Change Formula

Consider a single swap (Section 6) performed over an object $i \in [d]$ and two cliques $A, B \in \mathcal{Q}$, yielding a new solution $\mathcal{Q}'$. The induced change in objective value is given by $C(\mathcal{Q}') - C(\mathcal{Q})$, which expands to

$$\left[\sum_{Q,R\in\mathcal{Q}'} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q}\right] - \left[\sum_{Q,R\in\mathcal{Q}} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q}\right]. \tag{13}$$

All cost terms between cliques $Q, R \in (\mathcal{Q} \cap \mathcal{Q}')$ are present in both solution and cancel out. We therefore only consider cost terms that involve at least one of the two cliques $\{A, B\} \subseteq (\mathcal{Q})$ or respectively $\{A', B'\} \subseteq (\mathcal{Q}')$:

$$\begin{aligned}\sum_{Q,R\in\mathcal{Q}} \cdot = \sum_{Q\in\mathcal{Q}} \sum_{R\in\mathcal{Q}} \cdot &= \left(\sum_{Q\in\{A,B\}} \sum_{R\in\mathcal{Q}} \cdot\right) + \left(\sum_{Q\in\mathcal{Q}\setminus\{A,B\}} \sum_{R\in\mathcal{Q}} \cdot\right) \\ &= \left(\sum_{Q\in\{A,B\}} \sum_{R\in\mathcal{Q}} \cdot\right) + \left(\sum_{Q\in\mathcal{Q}\setminus\{A,B\}} \sum_{R\in\{A,B\}} \cdot\right) \\ &\quad + \left(\sum_{Q\in\mathcal{Q}\setminus\{A,B\}} \sum_{R\in\mathcal{Q}\setminus\{A,B\}} \cdot\right).\end{aligned} \tag{14}$$

The last term cancels in $C(\mathcal{Q}') - C(\mathcal{Q})$:

$$\begin{aligned}C(\mathcal{Q}') - C(\mathcal{Q}) = &\left[\sum_{Q\in\{A',B'\}} \sum_{R\in\mathcal{Q}'} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q}\right] \\ &+ \left[\sum_{Q\in\mathcal{Q}'\setminus\{A',B'\}} \sum_{R\in\{A',B'\}} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q}\right] \\ &- \left[\sum_{Q\in\{A,B\}} \sum_{R\in\mathcal{Q}} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q}\right] \\ &- \left[\sum_{Q\in\mathcal{Q}\setminus\{A,B\}} \sum_{R\in\{A,B\}} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q}\right].\end{aligned} \tag{15}$$

Apart from being limited to two cliques A, B, a single swap is also restricted to a single object $i \in [d]$. It follows, that all cost terms not involving i also cancel out.

Further fixing another object $j \in [d] \setminus \{i\}$, we define the change in cost restricted to i and any other object j as $\delta C^{i,j}_{\text{swap}}(A, B)$:

$$\begin{aligned}
\delta C^{i,j}_{\text{swap}}(A,B) :- &\left[\sum_{Q\in\{A',B'\}}\sum_{R\in\mathcal{Q}'}\sum_{\substack{p\in[D(Q)\cap D(R)\cap\{i\}]\\ q\in[D(Q)\cap D(R)\cap\{j\}]}} C^{p,q}_{Q^pQ^q,R^pR^q} + C^{q,p}_{Q^pQ^q,R^pR^q}\right] \\
&+\left[\sum_{Q\in\mathcal{Q}'\setminus\{A',B'\}}\sum_{R\in\{A',B'\}}\sum_{\substack{p\in[D(Q)\cap D(R)\cap\{i\}]\\ q\in[D(Q)\cap D(R)\cap\{j\}]}} C^{p,q}_{Q^pQ^q,R^pR^q} + C^{q,p}_{Q^pQ^q,R^pR^q}\right] \\
&-\left[\sum_{Q\in\{A,B\}}\sum_{R\in\mathcal{Q}}\sum_{\substack{p\in[D(Q)\cap D(R)\cap\{i\}]\\ q\in[D(Q)\cap D(R)\cap\{j\}]}} C^{p,q}_{Q^pQ^q,R^pR^q} + C^{q,p}_{Q^pQ^q,R^pR^q}\right] \\
&-\left[\sum_{Q\in\mathcal{Q}\setminus\{A,B\}}\sum_{R\in\{A,B\}}\sum_{\substack{p\in[D(Q)\cap D(R)\cap\{i\}]\\ q\in[D(Q)\cap D(R)\cap\{j\}]}} C^{p,q}_{Q^pQ^q,R^pR^q} + C^{q,p}_{Q^pQ^q,R^pR^q}\right].
\end{aligned} \tag{16}$$

Using Eq. (16), we can write the change in objective as a sum over all other objects $j \in [d] \setminus \{i\}$:

$$C(\mathcal{Q}') - C(\mathcal{Q}) = \sum_{j\in[d]\setminus\{i\}} \delta C^{i,j}_{\text{swap}}(A,B)\,, \tag{17}$$

which is the formulation of Eq. (4).

D Proof for Polynomial Reduction from Incomplete to Complete MGM

Before we prove the polynomial-time reduction from incomplete to complete MGM, we formalize complete MGM analogously to incomplete MGM in Section 3, denoting complete pendants to incomplete quantities with a tilde, *e.g.*, V^p and $\widetilde{V}^p$.

Instances, hereafter referred to as problems, of incomplete (or complete) problems are fully defined via their objects V^p, $p \in [d]$, (or $\widetilde{V}^p$, $p \in [\widetilde{d}]$,) and costs $C^{p,q}$, $p, q \in [d]$, (or $\widetilde{C}^{p,q}$, $p, q \in [\widetilde{d}]$,) with the difference that objects of the complete instance must have equal cardinalities $\widetilde{V}^p = \widetilde{V}^q =: \widetilde{n} \quad \forall p, q \in [\widetilde{d}]$. We denote the set of feasible vertex partitions or solutions to the complete problem with $\widetilde{\mathbb{Q}}$. All cliques $\widetilde{Q} \in \widetilde{\mathcal{Q}}$ of solutions $\widetilde{\mathcal{Q}} \in \widetilde{\mathbb{Q}}$ to the complete problem must contain *exactly one* element per object $p \in [\widetilde{d}]$, *i.e.*, $|\widetilde{Q} \cap \widetilde{V}^p| = 1$, which implies that cliques are *maximal* $D(\widetilde{Q}) = [\widetilde{d}]$. Furthermore, since all vertices must be matched, solutions $\widetilde{\mathcal{Q}} \in \widetilde{\mathbb{Q}}$ contain exactly $\widetilde{n}$ cliques $|\widetilde{\mathcal{Q}}| = \widetilde{n}$. The objective for complete MGM is defined in analogy to Eq. (1)

$$\min_{\widetilde{\mathcal{Q}}\in\widetilde{\mathbb{Q}}} \left[\widetilde{C}(\widetilde{\mathcal{Q}}) := \sum_{\widetilde{Q},\widetilde{R}\in\widetilde{\mathcal{Q}}}\sum_{p,q\in[\widetilde{d}]} \widetilde{C}^{p,q}_{\widetilde{Q}^p\widetilde{Q}^q,\widetilde{R}^p\widetilde{R}^q}\right]. \tag{18}$$

Proof of Theorem 1. As described in Eq. (9) and (10), given an incomplete problem with objects V^p, $p \in [d]$, and costs $C^{p,q}$, $p, q \in [d]$, we construct a complete problem with objects

$$\widetilde{V}^p = V^p \cup \Delta^p, \quad p \in [d]\,, \tag{19}$$

and costs

$$\widetilde{C}^{p,q}_{is,jt} = \begin{cases} C^{p,q}_{is,jt} & \text{, if } i,s,j,t \in V \\ 0 & \text{, else} \end{cases}, \tag{20}$$

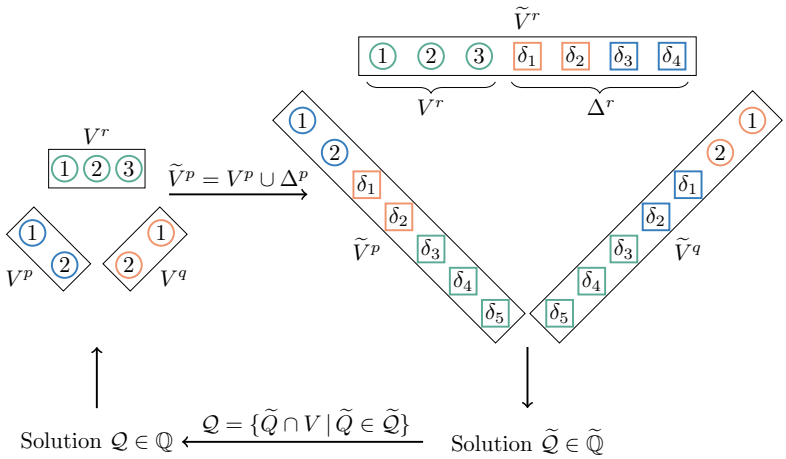


Figure D.1: **Reducing Incomplete to Complete MGM.** To reduce an incomplete problem with objects V^p, V^q, V^r to a complete problem, one adds dummy vertices Δ^p to each object V^p. Solutions to the incomplete problem can be recovered from solutions to the complete one by removing dummy vertices from cliques. However, this transformation is, without further reasoning, *impractical* since *each* object $\widetilde{V}^p$ of the complete problem is the same size as the *entire* incomplete problem $|V|$.

where Δ^p, $p \in [d]$ are *dummy vertices s.t.* $|\Delta^p| = |V| - |V^p|$, see Fig. D.1. This problem is indeed *complete* since $|\widetilde{V}^p| = |V^p| + |V| - |V^p| = |V| \quad \forall p \in [d]$. It is also constructed in *polynomial time* since $\sum_{p\in[d]} |V| - |V^p| = (d-1)|V|$ dummy vertices and $\sum_{p\in[d]} \sum_{q\in[d]\setminus\{p\}} (|V|^4 - |V^p|^2|V^q|^2)$ costs are added to the incomplete problem.

What remains to prove is that the optimal solution of the incomplete problem can be determined in polynomial time by solving the complete one. This follows from the fact that we can *translate* solutions without changing their objective between the incomplete and complete problem by *adding* and *removing* dummy vertices. A solution $\widetilde{\mathcal{Q}} \in \widetilde{\mathbb{Q}}$ for the complete problem is translated to the solution

$$\mathcal{Q} := \{\widetilde{Q} \cap V \mid \widetilde{Q} \in \widetilde{\mathcal{Q}}\}\,, \tag{21}$$

for the incomplete problem. We relate the cliques of both solutions via the *well-defined* function $\tau : \widetilde{\mathcal{Q}} \to \mathcal{Q}, \widetilde{Q} \mapsto \widetilde{Q} \cap V$. The solution to the incomplete problem is actually a solution, *i.e.*, $\mathcal{Q} \in \mathbb{Q}$, because it only contains *non-dummy vertices* by construction and *exhaustiveness*, *pairwise disjointness*, and *feasibility* carry over from the solution to the complete problem. Moreover, both objectives are equal

$$\begin{aligned}
\widetilde{C}(\widetilde{\mathcal{Q}}) &= \textstyle\sum_{\widetilde{Q},\widetilde{R}\in\widetilde{\mathcal{Q}}} \sum_{p,q\in[d]} \widetilde{C}^{p,q}_{\widetilde{Q}^p\widetilde{Q}^q,\widetilde{R}^p\widetilde{R}^q} \\
&\overset{\text{Eq. (20)}}{=} \textstyle\sum_{\widetilde{Q},\widetilde{R}\in\widetilde{\mathcal{Q}}} \sum_{p,q\in D(\tau(\widetilde{Q}))\cap D(\tau(\widetilde{R}))} C^{p,q}_{\tau(\widetilde{Q})^p\tau(\widetilde{Q})^q,\tau(\widetilde{R})^p\tau(\widetilde{R})^q} \\
&= \textstyle\sum_{Q,R\in\mathcal{Q}} \sum_{p,q\in D(Q)\cap D(R)} C^{p,q}_{Q^pQ^q,R^pR^q} \\
&= C(\mathcal{Q})\,.
\end{aligned} \tag{22}$$

For the other direction, Algorithm D.1 can be used to translate an incomplete solution $\mathcal{R} \in \mathbb{Q}$ to a complete solution $\widetilde{\mathcal{R}} \in \widetilde{\mathbb{Q}}$, which is indeed a solution because each dummy vertex is added to *exactly one* (possibly empty) clique out of $|V|$ in total, which means cliques are *maximal* and *exhaustiveness*, *pairwise disjointness*, and *feasibility* carry over from the solution to the incomplete problem. Note, however, that this solution is generally not unique because the enumeration of dummy vertices in Algorithm D.1 is an unspecified degree of freedom. Now, it holds by construction that $\mathcal{R} = \{\widetilde{R} \cap V \mid \widetilde{R} \in \widetilde{\mathcal{R}}\}$, hence, $C(\mathcal{R}) = \widetilde{C}(\widetilde{\mathcal{R}})$ by Eq. (22).

Therefore, translating any optimal solution to the complete problem via Eq. (21) yields an optimal solution to the incomplete one, which concludes the proof because this translation can be computed in polynomial time.

□ □

Algorithm D.1: Incomplete to Complete Solution Translation

Input: Incomplete Solution $\mathcal{Q} \in \mathbb{Q}$

$\mathcal{Q} \leftarrow \mathcal{Q} \setminus \{\emptyset\}$

`// enumerate cliques`

$\{Q_l\}_{l \in [|\mathcal{Q}|]} \leftarrow \mathcal{Q}$

$\widetilde{\mathcal{Q}} \leftarrow \{\widetilde{Q}_l\}_{l \in [|V|]}$ where $\widetilde{Q}_l = Q_l$ if $l \leq |\mathcal{Q}|$ and $\widetilde{Q}_l = \emptyset$ else

for $p \in [d]$ **do**

  `// enumerate dummy vertices`

  $\{\delta_l\}_{l \in [|V| - |V^p|]} \leftarrow \Delta^p$

  Reorder $\widetilde{\mathcal{Q}}$ *s.t.* $|\widetilde{Q}_l \cap V^p| = 1$ if $l \leq |V^p|$

  for $l \in [|V| - |V^p|]$ **do**

    $\widetilde{Q}_{l+|V^p|} \leftarrow \widetilde{Q}_{l+|V^p|} \cup \delta_l$

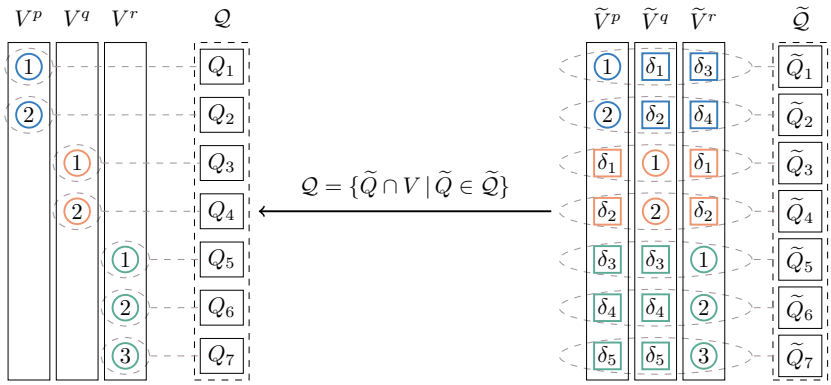


Figure D.2: **Requiring All Dummy Vertices.** In the reduction from Theorem 1, to obtain the incomplete solution that does not match any vertex (left), all dummy vertices are required by the complete solution (right). This example uses the setting of Fig. D.1. Solutions are illustrated identically to Fig. 3.

Minimality w.r.t. Number of Dummy Vertices

As discussed in Section 8, the number of dummy vertices required in Eq. (19) is $d-1$ times the size of the entire incomplete problem and yet *minimal* because the incomplete solution not matching any vertex can only be obtained through a complete solution matching each non-dummy vertex to $(d-1)$ dummy vertices, see Fig. D.2 for an example in the setting of Fig. D.1.

E Detailed Objective Values per Instance

In most cases the best and the second best results have been obtained by *Our(best)* and *Our(worst)* computed based on 10 runs of *Our* algorithm. Note that these runs can be executed in parallel in the same time and even their sequential execution often takes less time then a single execution of the competing algorithms. This, in turn, means that achieving even *Our(best)* results is computationally cheaper than execution of the competing algorithms.

Table E.1: **Synthetic complete (4 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-496.08	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
MP-T (Syn)	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
MP-T [44]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
MST [45]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
MST-Star [52]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Our (Syn)	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Our (best)	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Our (worst)	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Our-C (best)	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Our-C (worst)	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
PPI ($\alpha = -1$) [14]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
PPI ($\alpha = -10$) [14]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
PPI ($\alpha = -50$) [14]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
PPI ($\alpha = 0$) [14]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Stiefel [7]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**
Spectral [36, 9]	**-558.48**	**-546.60**	**-552.41**	**-559.55**	**-558.49**	**-553.06**	**-543.75**	**-547.87**	**-553.29**	**-554.76**

Table E.2: **Synthetic complete (8 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
MP-T (Syn)	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
MP-T [44]	-2514.68	**-2565.68**	-2490.44	-2393.16	**-2610.60**	-2464.22	-2429.58	-2456.84	**-2592.51**	**-2565.65**
MST [45]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
MST-Star [52]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Our (Syn)	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Our (best)	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Our (worst)	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Our-C (best)	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Our-C (worst)	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
PPI ($\alpha = -1$) [14]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
PPI ($\alpha = -10$) [14]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
PPI ($\alpha = -50$) [14]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
PPI ($\alpha = 0$) [14]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Stiefel [7]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**
Spectral [36, 9]	**-2587.65**	**-2565.68**	**-2585.59**	**-2562.17**	**-2610.60**	**-2593.88**	**-2558.20**	**-2583.81**	**-2592.51**	**-2565.65**

Table E.3: **Synthetic complete (12 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
MP-T (Syn)	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
MP-T [44]	-5714.89	**-6039.32**	-5958.21	-5784.81	-5911.11	-5998.88	**-6012.00**	**-6082.33**	**-6089.19**	-5887.15
MST [45]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
MST-Star [52]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Our (Syn)	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Our (best)	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Our (worst)	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Our-C (best)	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Our-C (worst)	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
PPI ($\alpha = -1$) [14]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
PPI ($\alpha = -10$) [14]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
PPI ($\alpha = -50$) [14]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
PPI ($\alpha = 0$) [14]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Stiefel [7]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**
Spectral [36, 9]	**-6092.97**	**-6039.32**	**-6107.78**	**-6052.55**	**-6121.11**	**-6109.68**	**-6012.00**	**-6082.33**	**-6089.19**	**-6058.51**

Table E.4: **Synthetic complete (16 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
MP-T (Syn)	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
MP-T [44]	**-11080.96**	**-11021.58**	-10614.98	-10521.96	**-11142.22**	-10815.80	**-10965.60**	-10478.03	**-11095.05**	-10696.66
MST [45]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
MST-Star [52]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Our (Syn)	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Our (best)	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Our (worst)	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Our-C (best)	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Our-C (worst)	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
PPI ($\alpha = -1$) [14]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
PPI ($\alpha = -10$) [14]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
PPI ($\alpha = -50$) [14]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
PPI ($\alpha = 0$) [14]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Stiefel [7]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**
Spectral [36, 9]	**-11080.96**	**-11021.58**	**-11059.12**	**-10953.02**	**-11142.22**	**-11079.59**	**-10965.60**	**-11096.36**	**-11095.05**	**-10995.11**

Table E.5: **Synthetic deform (4 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-300.55	-258.25	-201.20	**-335.48**	-229.73	-216.31	-252.86	-272.90	-256.74	**-333.06**
MP-T (Syn)	-241.58	-257.81	-179.15	-276.03	-176.16	-255.54	-248.76	-189.25	-212.91	-331.06
MP-T [44]	-259.75	-243.01	-221.72	-273.21	-244.24	-253.33	-249.54	-261.09	-272.10	-251.08
MST [45]	-323.76	-296.22	-286.16	-307.18	-300.12	-308.94	-313.07	-312.51	-302.27	-331.06
MST-Star [52]	-323.76	-302.02	-275.60	-323.77	-307.08	-308.94	-310.72	-317.30	-302.27	-312.95
Our (Syn)	-289.39	-312.33	-228.19	-289.11	-244.96	-300.97	-297.51	-300.08	-272.78	-325.36
Our (best)	**-342.83**	**-321.97**	**-293.32**	-332.71	**-320.10**	**-316.27**	**-327.82**	**-337.11**	**-310.94**	**-333.06**
Our (worst)	-320.39	-316.59	-275.08	-325.74	-304.48	-310.91	-313.53	-316.47	-293.91	-308.68
Our-C (best)	**-342.83**	**-321.97**	-290.45	-332.71	-313.84	-315.79	**-327.82**	-336.97	-307.92	**-333.06**
Our-C (worst)	-312.79	-292.71	-268.69	-318.48	-297.31	-301.19	-306.26	-310.90	-288.37	-304.37
PPI ($\alpha = -1$) [14]	-239.50	-275.12	-224.91	-285.34	-234.92	-287.56	-253.50	-263.17	-233.62	-325.36
PPI ($\alpha = -10$) [14]	-239.50	-275.12	-224.91	-285.34	-234.92	-287.56	-253.50	-263.17	-233.62	-325.36
PPI ($\alpha = -50$) [14]	-239.50	-275.12	-224.91	-285.34	-234.92	-287.56	-253.50	-263.17	-233.62	-325.36
PPI ($\alpha = 0$) [14]	-239.50	-275.12	-224.91	-285.34	-234.92	-287.56	-253.50	-263.17	-233.62	-325.36
Stiefel [7]	-246.20	-308.17	-214.10	-281.79	-260.42	-299.52	-271.89	-273.62	-275.13	-331.06
Spectral [36, 9]	-311.32	-305.97	-255.17	-288.32	-293.01	-301.13	-313.60	-323.15	-272.69	-331.06

Table E.6: **Synthetic deform (8 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-1359.40	-1451.00	-1102.54	-1381.85	**-1510.40**	-1438.76	-1379.50	-1518.18	**-1536.48**	-1473.00
MP-T (Syn)	-1073.28	-1263.18	-1091.96	-1129.46	-1100.77	-960.39	-1297.39	-1076.83	-1302.86	-1047.33
MP-T [44]	-1077.48	-1073.27	-1038.39	-1091.66	-1056.98	-1094.43	-1126.61	-1124.24	-1143.33	-1065.24
MST [45]	-1428.33	-1380.78	-1205.17	-1308.09	-1259.18	-1348.89	-1414.29	-1268.09	-1466.25	-1378.46
MST-Star [52]	-1428.33	-1368.54	-1244.92	-1304.21	-1343.51	-1401.00	-1427.70	-1407.45	-1437.75	-1384.39
Our (Syn)	-1324.95	-1306.63	-1092.79	-1344.98	-1203.89	-1124.11	-1280.34	-1157.63	-1410.22	-1306.23
Our (best)	**-1523.81**	**-1456.37**	**-1400.88**	**-1458.59**	**-1510.40**	**-1497.54**	**-1501.71**	**-1544.77**	**-1536.48**	**-1484.89**
Our (worst)	**-1523.81**	-1352.34	-1316.56	-1421.30	-1350.41	-1411.94	**-1501.71**	-1383.75	-1455.26	-1395.26
Our-C (best)	**-1523.81**	-1451.67	-1399.80	**-1458.59**	**-1510.40**	-1454.10	**-1501.71**	**-1544.77**	-1524.00	-1447.43
Our-C (worst)	-1332.59	-1316.75	-1250.74	-1309.20	-1304.83	-1314.44	-1348.24	-1334.56	-1302.36	-1303.50
PPI ($\alpha = -1$) [14]	-1206.75	-1320.08	-1002.31	-1170.86	-1141.62	-1149.83	-1287.75	-1084.23	-1369.18	-1234.06
PPI ($\alpha = -10$) [14]	-1206.75	-1320.08	-1002.31	-1170.86	-1141.62	-1149.83	-1287.75	-1084.23	-1369.18	-1234.06
PPI ($\alpha = -50$) [14]	-1206.75	-1320.08	-1002.31	-1170.86	-1141.62	-1149.83	-1287.75	-1084.23	-1369.18	-1234.06
PPI ($\alpha = 0$) [14]	-1206.75	-1320.08	-1002.31	-1170.86	-1141.62	-1149.83	-1287.75	-1084.23	-1369.18	-1234.06
Stiefel [7]	-1290.64	-1225.39	-1041.66	-1257.69	-1301.75	-1139.65	-1282.15	-1166.61	-1421.89	-1259.67
Spectral [36, 9]	-1039.06	-1278.72	-1134.08	-1254.22	-1249.44	-1138.73	-1350.71	-1238.23	-1340.46	-1274.97

Table E.7: **Synthetic deform (12 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-3472.17	-3390.14	-3313.19	**-3467.06**	**-3498.74**	-3423.23	-3412.43	-3546.99	-3519.20	-3489.32
MP-T (Syn)	-2950.76	-3227.93	-2607.77	-2663.14	-3419.32	-2811.92	-3336.88	-3237.99	-3121.32	-3352.25
MP-T [44]	-2501.12	-2553.97	-2463.41	-2502.23	-2338.36	-2531.84	-2623.14	-2587.31	-2657.60	-2504.11
MST [45]	-3158.22	-3004.91	-2745.91	-2939.39	-2833.36	-3090.51	-3311.39	-3405.55	-3143.44	-3180.28
MST-Star [52]	-3285.58	-3192.73	-2955.23	-3023.61	-2987.80	-3266.58	-3342.76	-3406.88	-3221.82	-3286.74
Our (Syn)	-3160.62	-3154.82	-2425.63	-2966.11	-3419.32	-2516.86	-3360.00	-3218.31	-3133.52	-3121.60
Our (best)	**-3642.08**	**-3477.52**	**-3318.36**	**-3467.06**	**-3498.74**	**-3478.99**	**-3480.48**	**-3650.50**	**-3522.58**	**-3520.58**
Our (worst)	**-3642.08**	**-3477.52**	-3030.20	-3465.70	**-3498.74**	-3147.24	**-3480.48**	-3403.00	-3271.79	**-3520.58**
Our-C (best)	**-3642.08**	-3472.93	-3249.69	-3405.63	-3405.38	-3219.17	-3472.07	**-3650.50**	-3514.62	-3509.51
Our-C (worst)	-3328.76	-2965.79	-2909.93	-3033.99	-2973.73	-3050.96	-2971.48	-3101.67	-3084.10	-3283.28
PPI ($\alpha = -1$) [14]	-3032.24	-3223.24	-2518.60	-2816.83	-3036.67	-2745.14	-2947.76	-3033.10	-2816.07	-3193.91
PPI ($\alpha = -10$) [14]	-3032.24	-3223.24	-2518.60	-2816.83	-3036.67	-2745.14	-2947.76	-3033.10	-2816.07	-3193.91
PPI ($\alpha = -50$) [14]	-3032.24	-3223.24	-2518.60	-2816.83	-3036.67	-2745.14	-2947.76	-3033.10	-2816.07	-3193.91
PPI ($\alpha = 0$) [14]	-3032.24	-3223.24	-2518.60	-2816.83	-3036.67	-2745.14	-2947.76	-3033.10	-2816.07	-3193.91
Stiefel [7]	-3115.34	-3260.37	-2489.57	-2995.29	-3330.71	-2680.03	-3198.72	-3244.04	-3065.90	-3175.29
Spectral [36, 9]	-3100.72	-3270.11	-2562.64	-2841.84	-3156.92	-2520.59	-3105.31	-2849.44	-3086.78	-3177.61

Table E.8: **Synthetic deform (16 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-6102.13	**-6569.81**	-6150.22	-6267.58	**-6272.24**	-6042.47	-5603.83	-6462.57	-6410.54	-6552.52
MP-T (Syn)	-5536.31	-6476.52	-4661.78	-5418.13	-6044.64	-4941.69	-5424.98	-6284.62	-5942.83	-6238.34
MP-T [44]	-4596.00	-4586.45	-4462.58	-4601.28	-4206.68	-4587.89	-4515.56	-4594.57	-4897.13	-4556.19
MST [45]	-5520.30	-6437.54	-5565.08	-5187.35	-5031.44	-5386.20	-5822.75	-6157.15	-5563.87	-5746.87
MST-Star [52]	-5825.20	-6304.99	-5568.92	-5583.91	-5407.07	-5733.57	-5955.77	-5853.55	-5803.71	-5826.07
Our (Syn)	-5754.77	-6441.87	-4154.29	-5165.15	-6115.87	-5135.87	-5740.21	-6284.62	-5691.34	-6281.96
Our (best)	**-6507.40**	**-6569.81**	**-6151.96**	**-6278.85**	**-6272.24**	**-6349.59**	**-6231.85**	**-6615.35**	**-6411.24**	**-6561.48**
Our (worst)	-6491.59	**-6569.81**	**-6151.96**	-6266.32	-5351.73	-5797.67	-6231.65	-5572.09	-5660.36	**-6561.48**
Our-C (best)	-6486.80	**-6569.81**	-5996.61	-6130.09	-6128.86	-6156.88	-6119.53	**-6615.35**	-6400.92	-6550.48
Our-C (worst)	-5465.99	-5822.27	-5233.77	-5448.51	-4923.51	-5419.15	-5338.57	-5385.89	-5471.18	-5728.51
PPI ($\alpha = -1$) [14]	-5343.40	-6449.73	-4713.40	-4966.05	-6040.01	-4831.64	-5305.82	-5405.26	-5747.75	-6285.36
PPI ($\alpha = -10$) [14]	-5343.40	-6449.73	-4713.40	-4966.05	-6040.01	-4831.64	-5305.82	-5405.26	-5747.75	-6285.36
PPI ($\alpha = -50$) [14]	-5343.40	-6449.73	-4713.40	-4966.05	-6040.01	-4831.64	-5305.82	-5405.26	-5747.75	-6285.36
PPI ($\alpha = 0$) [14]	-5343.40	-6449.73	-4713.40	-4966.05	-6040.01	-4831.64	-5305.82	-5405.26	-5747.75	-6285.36
Stiefel [7]	-5476.12	-6376.02	-4817.78	-5288.46	-6152.68	-4894.31	-5798.80	-6229.24	-5632.25	-6158.10
Spectral [36, 9]	-5156.91	-6418.48	-4710.80	-5116.72	-5398.72	-4808.77	-5721.31	-5519.21	-5406.90	-6120.15

Table E.9: **Synthetic density (4 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-127.32	-202.04	-195.01	-225.28	-128.95	-152.09	-142.68	-179.58	-204.61	-196.60
MP-T (Syn)	-128.22	**-222.06**	-166.40	-219.06	-112.93	-200.77	-184.80	-216.94	-152.35	**-232.46**
MP-T [44]	-126.18	-168.00	-162.30	-175.28	-131.48	-161.30	-170.17	-165.38	-159.68	-193.65
MST [45]	-175.15	-208.46	-190.68	-228.04	-159.15	-208.36	-182.65	-214.93	-195.66	-228.02
MST-Star [52]	-170.51	-207.94	-186.90	-219.11	-167.78	-211.72	-193.38	-214.93	-195.66	-228.02
Our (Syn)	-152.90	**-222.06**	-175.34	-219.06	-135.14	-200.77	-182.79	-216.94	-175.63	**-232.46**
Our (best)	**-179.63**	**-222.06**	**-202.26**	**-232.06**	**-176.74**	**-213.24**	**-208.91**	**-228.06**	**-210.06**	**-232.46**
Our (worst)	-170.13	-206.80	-200.06	**-232.06**	-163.05	-197.62	-194.49	-221.14	-204.68	-226.77
Our-C (best)	-175.86	-219.83	**-202.26**	**-232.06**	-172.82	-208.75	-198.67	**-228.06**	-204.68	**-232.46**
Our-C (worst)	-161.36	-200.74	-193.04	-200.97	-162.69	-193.59	-193.86	-213.62	-190.67	-224.44
PPI ($\alpha = -1$) [14]	-111.38	-197.04	-177.46	-219.11	-115.70	-190.07	-159.78	-199.60	-170.95	**-232.46**
PPI ($\alpha = -10$) [14]	-111.38	-197.04	-177.46	-219.11	-115.70	-190.07	-159.78	-199.60	-170.95	**-232.46**
PPI ($\alpha = -50$) [14]	-111.38	-197.04	-177.46	-219.11	-115.70	-190.07	-159.78	-199.60	-170.95	**-232.46**
PPI ($\alpha = 0$) [14]	-111.38	-197.04	-177.46	-219.11	-115.70	-190.07	-159.78	-199.60	-170.95	**-232.46**
Stiefel [7]	-124.74	-190.48	-169.00	-200.78	-136.40	-190.07	-197.18	-216.94	-163.16	**-232.46**
Spectral [36, 9]	-137.02	**-222.06**	-185.66	-202.48	-148.84	-191.40	-193.77	-212.34	-184.66	**-232.46**

Table E.10: **Synthetic density (8 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-759.03	-969.81	**-933.74**	-790.95	-773.20	-716.29	-799.89	**-1062.28**	-771.63	-772.49
MP-T (Syn)	-644.19	-930.85	-857.59	-861.38	-643.25	-524.84	-868.82	**-1062.28**	-851.67	-915.20
MP-T [44]	-549.26	-750.97	-685.10	-689.82	-596.83	-616.10	-646.45	-750.82	-649.43	-737.14
MST [45]	-731.81	-945.83	-887.15	-851.44	-766.22	-821.40	-884.47	**-1062.28**	-790.34	-959.03
MST-Star [52]	-743.65	-933.68	-873.91	-842.11	-770.37	-820.22	-889.44	-1006.82	-847.90	-946.41
Our (Syn)	-592.46	-916.15	-708.97	-810.21	-621.26	-629.43	-856.30	**-1062.28**	-855.38	-941.73
Our (best)	**-788.79**	**-970.22**	**-933.74**	**-920.62**	**-816.22**	**-867.25**	**-956.28**	**-1062.28**	**-938.28**	**-990.28**
Our (worst)	-737.58	**-970.22**	-847.47	-807.62	-753.24	-814.47	-895.38	**-1062.28**	-846.58	-952.76
Our-C (best)	-783.28	-969.81	-904.70	-919.80	-792.82	-854.36	-928.66	**-1062.28**	**-938.28**	**-990.28**
Our-C (worst)	-706.34	-818.47	-807.87	-807.62	-706.62	-786.88	-833.84	-907.88	-806.48	-856.05
PPI ($\alpha = -1$) [14]	-547.99	-930.85	-751.57	-834.28	-619.45	-682.50	-717.21	**-1062.28**	-710.25	-869.19
PPI ($\alpha = -10$) [14]	-547.99	-930.85	-751.57	-834.28	-619.45	-682.50	-717.21	**-1062.28**	-710.25	-869.19
PPI ($\alpha = -50$) [14]	-547.99	-930.85	-751.57	-834.28	-619.45	-682.50	-717.21	**-1062.28**	-710.25	-869.19
PPI ($\alpha = 0$) [14]	-547.99	-930.85	-751.57	-834.28	-619.45	-682.50	-717.21	**-1062.28**	-710.25	-869.19
Stiefel [7]	-574.45	-930.85	-830.38	-847.61	-636.18	-638.44	-800.21	**-1062.28**	-803.97	-946.36
Spectral [36, 9]	-586.56	-930.85	-776.62	-853.17	-617.23	-669.89	-853.03	**-1062.28**	-823.40	-916.63

Table E.11: **Synthetic density (12 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-1930.30	**-2326.68**	-2044.00	-2174.62	-1828.16	-1938.15	**-2162.66**	**-2374.66**	**-2298.66**	**-2350.66**
MP-T (Syn)	-1518.26	**-2326.68**	-1801.83	-2057.73	-1525.98	-1421.54	-2142.07	**-2374.66**	-2236.16	-2274.76
MP-T [44]	-1312.45	-1685.81	-1554.37	-1487.44	-1444.97	-1422.00	-1462.78	-1522.89	-1523.21	-1785.80
MST [45]	-1716.74	-2122.97	-1925.06	-1970.40	-1875.20	-1904.95	-1874.35	-2283.46	-1933.33	-2291.76
MST-Star [52]	-1759.23	-2253.90	-1892.04	-2004.62	-1847.32	-1949.72	-2009.43	-2263.79	-2025.74	-2223.52
Our (Syn)	-1646.44	**-2326.68**	-1777.96	-2146.04	-1794.02	-1653.30	-1921.43	**-2374.66**	-2236.16	-2310.37
Our (best)	**-1941.46**	**-2326.68**	**-2075.42**	**-2174.66**	**-2006.66**	**-2013.55**	**-2162.66**	**-2374.66**	**-2298.66**	**-2350.66**
Our (worst)	-1751.36	**-2326.68**	-1904.04	**-2174.66**	-1791.08	-1848.43	**-2162.66**	**-2374.66**	**-2298.66**	**-2350.66**
Our-C (best)	-1935.62	**-2326.68**	-1997.62	-2104.45	-1980.57	-1992.10	-2154.84	**-2374.66**	-2265.93	**-2350.66**
Our-C (worst)	-1644.13	-2018.96	-1819.23	-1788.94	-1706.30	-1775.49	-1885.24	-2203.97	-1895.58	-1920.12
PPI ($\alpha = -1$) [14]	-1436.70	**-2326.68**	-1624.70	-2024.10	-1570.67	-1477.44	-1944.32	-2353.32	-2214.01	-2308.66
PPI ($\alpha = -10$) [14]	-1436.70	**-2326.68**	-1624.70	-2024.10	-1570.67	-1477.44	-1944.32	-2353.32	-2214.01	-2308.66
PPI ($\alpha = -50$) [14]	-1436.70	**-2326.68**	-1624.70	-2024.10	-1570.67	-1477.44	-1944.32	-2353.32	-2214.01	-2308.66
PPI ($\alpha = 0$) [14]	-1436.70	**-2326.68**	-1624.70	-2024.10	-1570.67	-1477.44	-1944.32	-2353.32	-2214.01	-2308.66
Stiefel [7]	-1654.88	-2274.60	-1727.16	-2029.04	-1674.76	-1566.04	-2128.23	**-2374.66**	-2233.78	-2321.22
Spectral [36, 9]	-1589.40	-2274.60	-1657.32	-1979.28	-1605.12	-1490.94	-2103.12	**-2374.66**	-2236.16	-2284.01

Table E.12: **Synthetic density (16 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
DS* [8]	-3611.85	**-4271.20**	**-3855.68**	-3672.47	**-3599.20**	-3501.50	**-3878.61**	**-4235.20**	**-4319.20**	**-4203.20**
MP-T (Syn)	-3401.30	**-4271.20**	-3309.30	-3834.94	-3353.09	-2751.06	-3824.66	**-4235.20**	-4308.86	-4123.88
MP-T [44]	-2415.26	-3129.29	-2794.45	-2769.06	-2514.60	-2489.36	-2568.80	-2698.16	-2872.80	-2949.04
MST [45]	-3214.17	-3601.05	-3560.12	-3552.12	-3387.10	-3080.99	-3424.63	-4050.58	-4202.64	-4071.88
MST-Star [52]	-3251.09	-3929.68	-3505.68	-3594.58	-3227.59	-3458.98	-3610.43	-3993.48	-4129.00	-3977.67
Our (Syn)	-3324.64	**-4271.20**	-3403.10	-3904.59	-3491.43	-3045.23	-3835.58	**-4235.20**	-4308.86	-4178.04
Our (best)	**-3615.19**	**-4271.20**	**-3855.68**	**-3934.51**	**-3599.20**	**-3655.20**	**-3878.61**	**-4235.20**	**-4319.20**	**-4203.20**
Our (worst)	-3166.69	**-4271.20**	-3441.45	**-3934.51**	-3235.72	-3237.90	-3449.02	**-4235.20**	**-4319.20**	**-4203.20**
Our-C (best)	-3595.18	-4235.60	-3622.02	-3854.66	-3488.16	-3496.59	-3875.20	**-4235.20**	**-4319.20**	**-4203.20**
Our-C (worst)	-3031.47	-3529.46	-3270.01	-3254.39	-3034.19	-3112.27	-3338.71	-3503.95	-3473.68	-3419.59
PPI ($\alpha = -1$) [14]	-3282.39	**-4271.20**	-3367.93	-3863.73	-2907.88	-2521.89	-3818.20	**-4235.20**	-4308.86	-4160.08
PPI ($\alpha = -10$) [14]	-3282.39	**-4271.20**	-3367.93	-3863.73	-2907.88	-2521.89	-3818.20	**-4235.20**	-4308.86	-4160.08
PPI ($\alpha = -50$) [14]	-3282.39	**-4271.20**	-3367.93	-3863.73	-2907.88	-2521.89	-3818.20	**-4235.20**	-4308.86	-4160.08
PPI ($\alpha = 0$) [14]	-3282.39	**-4271.20**	-3367.93	-3863.73	-2907.88	-2521.89	-3818.20	**-4235.20**	-4308.86	-4160.08
Stiefel [7]	-3236.71	-4231.16	-3285.32	-3844.63	-3118.98	-2865.77	-3840.05	-4220.86	-4277.02	-4178.04
Spectral [36, 9]	-3373.49	-4231.16	-3097.15	-3810.30	-3012.60	-3003.76	-3816.55	**-4235.20**	-4308.86	-4160.08

Table E.13: **Synthetic outlier (4 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	68.32	114.12	101.65	101.21	70.70	93.83	127.61	109.34	111.88	106.46
MP-T [44]	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
MST [45]	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
MST-Star [52]	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
Our (Syn)	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
Our (best)	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
Our (worst)	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
Our-C (best)	**-43.28**	**-43.22**	**-43.22**	**-43.28**	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
Our-C (worst)	-43.22	**-43.22**	**-43.22**	-43.22	**-43.80**	**-43.93**	**-43.22**	**-43.22**	**-43.22**	**-43.22**
PPI ($\alpha = -1$) [14]	151.31	123.79	126.37	114.25	91.40	138.68	107.06	115.97	109.48	95.50
PPI ($\alpha = -10$) [14]	151.31	123.79	126.37	114.25	91.40	138.68	107.06	115.97	109.48	95.50
PPI ($\alpha = -50$) [14]	151.31	123.79	126.37	114.25	91.40	138.68	107.06	115.97	109.48	95.50
PPI ($\alpha = 0$) [14]	151.31	123.79	126.37	114.25	91.40	138.68	107.06	115.97	109.48	95.50
Stiefel [7]	107.49	116.87	123.31	120.32	110.56	119.03	113.74	117.90	111.75	110.41
Spectral [36, 9]	59.17	72.86	59.69	31.01	37.28	67.33	76.65	62.67	64.79	70.23

Table E.14: **Synthetic outlier (8 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	542.36	538.54	540.90	512.37	434.00	378.01	548.63	345.53	478.42	403.78
MP-T [44]	-177.99	-118.01	-191.68	-163.66	-198.75	-36.83	-116.89	-164.24	-160.04	-165.19
MST [45]	**-202.26**	**-201.70**	**-202.40**	-202.19	**-202.67**	**-204.36**	**-203.10**	**-202.33**	**-201.70**	-201.70
MST-Star [52]	-202.21	**-201.70**	**-202.40**	-202.13	-202.28	-203.49	**-203.10**	**-202.33**	**-201.70**	-201.70
Our (Syn)	**-202.26**	**-201.70**	**-202.40**	-202.19	**-202.67**	**-204.36**	**-203.10**	**-202.33**	-199.04	-193.13
Our (best)	**-202.26**	**-201.70**	**-202.40**	**-202.31**	**-202.67**	**-204.36**	**-203.10**	**-202.33**	**-201.70**	**-201.75**
Our (worst)	**-202.26**	**-201.70**	**-202.40**	**-202.31**	**-202.67**	**-204.36**	**-203.10**	**-202.33**	**-201.70**	**-201.75**
Our-C (best)	-202.21	**-201.70**	**-202.40**	-202.26	**-202.67**	-204.21	**-203.10**	**-202.33**	**-201.70**	-201.70
Our-C (worst)	-202.21	**-201.70**	**-202.40**	-201.70	-202.28	-203.75	**-203.10**	**-202.33**	**-201.70**	-201.70
PPI ($\alpha = -1$) [14]	553.66	546.93	634.10	572.27	541.50	477.10	549.16	531.47	586.99	568.80
PPI ($\alpha = -10$) [14]	553.66	546.93	634.10	572.27	541.50	477.10	549.16	531.47	586.99	568.80
PPI ($\alpha = -50$) [14]	553.66	546.93	634.10	572.27	541.50	477.10	549.16	531.47	586.99	568.80
PPI ($\alpha = 0$) [14]	553.66	546.93	634.10	572.27	541.50	477.10	549.16	531.47	586.99	568.80
Stiefel [7]	513.21	538.86	532.00	523.60	493.64	406.92	574.01	530.65	627.48	718.77
Spectral [36, 9]	323.28	313.44	348.01	243.40	221.18	176.10	222.89	300.65	378.95	246.62

Table E.15: **Synthetic outlier (12 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	863.37	1040.37	641.11	726.51	1019.21	1030.27	1286.33	1216.33	1096.75	1298.77
MP-T [44]	-418.16	-450.80	-451.43	-423.76	-457.68	-221.10	-449.73	-429.57	-385.82	-371.59
MST [45]	**-476.28**	-475.44	-476.71	-474.23	-477.30	**-478.24**	**-477.09**	**-476.56**	**-475.89**	-475.44
MST-Star [52]	-475.95	-475.44	-476.14	-475.42	-476.13	-477.23	**-477.09**	-476.07	**-475.89**	-475.44
Our (Syn)	**-476.28**	-471.18	-469.63	-457.73	-477.30	**-478.24**	**-477.09**	**-476.56**	-473.23	-466.87
Our (best)	**-476.28**	**-475.47**	**-476.74**	**-476.47**	**-477.54**	**-478.24**	**-477.09**	**-476.56**	**-475.89**	**-475.49**
Our (worst)	**-476.28**	**-475.47**	**-476.74**	**-476.47**	**-477.54**	**-478.24**	**-477.09**	**-476.56**	**-475.89**	**-475.49**
Our-C (best)	-476.23	-475.44	-476.71	-476.29	-477.43	-477.79	**-477.09**	**-476.56**	**-475.89**	-475.44
Our-C (worst)	-475.95	-475.44	-476.14	-475.84	-477.04	-477.49	-476.84	-476.07	**-475.89**	-475.44
PPI ($\alpha = -1$) [14]	1320.46	1271.77	1219.17	1264.68	1284.87	1247.82	1253.24	1240.70	1280.23	1350.59
PPI ($\alpha = -10$) [14]	1320.46	1271.77	1219.17	1264.68	1284.87	1247.82	1253.24	1240.70	1280.23	1350.59
PPI ($\alpha = -50$) [14]	1320.46	1271.77	1219.17	1264.68	1284.87	1247.82	1253.24	1240.70	1280.23	1350.59
PPI ($\alpha = 0$) [14]	1320.46	1271.77	1219.17	1264.68	1284.87	1247.82	1253.24	1240.70	1280.23	1350.59
Stiefel [7]	1280.12	1540.30	1544.65	1191.32	1240.35	1184.11	1226.88	1203.56	1348.05	1575.70
Spectral [36, 9]	423.55	855.65	659.90	710.45	512.36	426.29	880.14	691.17	1049.95	683.05

Table E.16: **Synthetic outlier (16 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	1733.81	1552.57	1927.99	1925.45	1650.75	1614.19	1909.65	1561.19	2349.67	1409.16
MP-T [44]	-767.65	-698.38	-779.73	-815.39	-840.60	-482.10	-807.77	-783.80	-790.96	-689.79
MST [45]	**-866.71**	-866.70	-866.46	-865.24	-867.66	**-867.24**	**-866.42**	-866.27	**-864.88**	-867.00
MST-Star [52]	-865.26	-866.16	-865.18	-865.95	-865.71	-866.22	-866.09	-866.30	**-864.88**	-865.99
Our (Syn)	**-866.71**	-856.68	-859.38	-865.24	-867.04	**-867.24**	**-866.42**	-866.27	-862.23	-858.44
Our (best)	**-866.71**	**-866.75**	**-866.49**	**-867.03**	**-867.90**	**-867.24**	**-866.42**	**-867.69**	**-864.88**	**-867.05**
Our (worst)	**-866.71**	**-866.75**	**-866.49**	**-867.03**	**-867.90**	**-867.24**	**-866.42**	**-867.69**	**-864.88**	**-867.05**
Our-C (best)	-866.60	-866.70	-866.46	-866.80	-867.79	-866.64	-866.16	-867.42	**-864.88**	-867.00
Our-C (worst)	-865.54	-865.76	-865.89	-865.69	-866.59	-865.78	-865.84	-866.52	-864.43	-867.00
PPI ($\alpha = -1$) [14]	2313.98	2232.43	2239.33	2312.93	2207.55	2133.70	2331.69	2300.79	2284.11	2276.66
PPI ($\alpha = -10$) [14]	2313.98	2232.43	2239.33	2312.93	2207.55	2133.70	2331.69	2300.79	2284.11	2276.66
PPI ($\alpha = -50$) [14]	2313.98	2232.43	2239.33	2312.93	2207.55	2133.70	2331.69	2300.79	2284.11	2276.66
PPI ($\alpha = 0$) [14]	2313.98	2232.43	2239.33	2312.93	2207.55	2133.70	2331.69	2300.79	2284.11	2276.66
Stiefel [7]	2273.46	2787.82	2377.35	2276.56	2185.84	2196.56	2310.61	2261.99	2571.14	2639.39
Spectral [36, 9]	1186.75	1469.42	1240.60	1063.78	1067.85	1082.20	1328.48	1360.12	845.68	1888.81

Table E.17: **Hotel (4 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	172.92	166.88	118.55	46.48	200.55	202.04	132.08	71.46	115.36	145.93
MP-T [44]	-9.54	**-25.75**	**-25.06**	**-25.78**	-10.94	-6.67	**-20.20**	-25.98	-14.94	**-26.96**
MST [45]	-13.69	**-25.75**	-23.17	-23.92	-11.12	-6.49	**-20.20**	**-26.12**	-15.27	-24.60
MST-Star [52]	-13.69	**-25.75**	-22.94	-23.92	-10.36	-6.49	**-20.20**	**-26.12**	-11.94	-22.70
Our (Syn)	34.78	**-25.75**	-24.99	-25.44	34.61	12.43	-17.53	-16.76	34.02	**-26.96**
Our (best)	**-13.87**	**-25.75**	**-25.06**	**-25.78**	**-11.58**	**-8.10**	**-20.20**	**-26.12**	**-16.67**	**-26.96**
Our (worst)	-10.90	**-25.75**	**-25.06**	**-25.78**	-10.20	-6.86	**-20.20**	-22.63	-16.38	**-26.96**
Our-C (best)	**-13.87**	**-25.75**	**-25.06**	**-25.78**	-11.36	**-8.10**	**-20.20**	-25.98	**-16.67**	**-26.96**
Our-C (worst)	-9.21	**-25.75**	-23.36	-21.85	-10.20	-6.54	-11.44	-16.07	-9.41	**-26.96**
PPI ($\alpha = -1$) [14]	267.09	201.60	177.05	204.26	302.49	284.35	146.02	146.19	247.50	183.10
PPI ($\alpha = -10$) [14]	267.09	201.60	177.05	204.26	302.49	284.35	146.02	146.19	247.50	183.10
PPI ($\alpha = -50$) [14]	267.09	201.60	177.05	204.26	302.49	284.35	146.02	146.19	247.50	183.10
PPI ($\alpha = 0$) [14]	267.09	201.60	177.05	204.26	302.49	284.35	146.02	146.19	247.50	183.10
Stiefel [7]	250.43	168.45	165.85	147.66	250.39	237.95	235.13	152.52	224.35	147.68
Spectral [36, 9]	157.07	138.86	124.54	86.58	200.18	197.37	227.20	83.68	102.20	57.94

Table E.18: **Hotel (8 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	374.82	497.75	472.98	293.62	592.68	374.78	358.32	474.01	453.47	432.01
MP-T [44]	-70.24	**-147.92**	-72.00	-88.25	-43.82	-58.13	-100.78	-55.59	-44.61	**-145.83**
MST [45]	-87.15	-144.94	-72.17	-71.43	-34.63	-43.26	-97.64	-58.80	-48.29	-131.77
MST-Star [52]	-83.58	-137.08	-63.03	-74.19	-46.86	-57.42	-92.04	-50.30	-54.71	-120.12
Our (Syn)	41.43	-146.05	-68.56	46.77	33.16	18.28	-56.01	242.12	74.78	**-145.83**
Our (best)	**-97.33**	**-147.92**	**-81.10**	**-96.33**	**-61.84**	**-69.24**	**-101.97**	**-81.95**	**-65.87**	**-145.83**
Our (worst)	**-97.33**	**-147.92**	-74.12	-85.84	-44.30	-60.78	-85.07	-62.22	-37.57	**-145.83**
Our-C (best)	-97.20	**-147.92**	-77.98	-94.99	-57.50	-63.07	**-101.97**	-78.86	-51.56	**-145.83**
Our-C (worst)	-64.27	-147.50	-49.91	-57.91	-27.31	-51.41	-83.91	-52.09	-32.04	-135.25
PPI ($\alpha = -1$) [14]	735.56	871.59	861.77	986.47	850.57	800.57	728.11	834.52	998.48	705.68
PPI ($\alpha = -10$) [14]	735.56	871.59	861.77	986.47	850.57	800.57	728.11	834.52	998.48	705.68
PPI ($\alpha = -50$) [14]	735.56	871.59	861.77	986.47	850.57	800.57	728.11	834.52	998.48	705.68
PPI ($\alpha = 0$) [14]	735.56	871.59	861.77	986.47	850.57	800.57	728.11	834.52	998.48	705.68
Stiefel [7]	910.64	727.24	994.98	869.29	947.13	1120.48	876.04	983.31	925.62	624.15
Spectral [36, 9]	488.61	589.10	538.22	628.80	706.74	684.14	230.58	726.49	891.15	202.24

Table E.19: **Hotel (12 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	591.38	863.61	596.74	983.26	755.49	1061.23	700.49	362.46	611.64	563.21
MP-T [44]	-157.08	**-341.76**	-185.34	-188.09	-95.82	-101.53	**-277.02**	-239.21	-143.85	**-327.81**
MST [45]	-128.48	-313.77	-171.91	-163.12	-59.77	-26.70	-256.78	-188.91	-111.27	-292.38
MST-Star [52]	-168.15	-299.59	-161.34	-171.24	-60.56	-87.77	-235.31	-183.26	-135.21	-272.65
Our (Syn)	210.82	-337.10	-55.55	166.80	295.47	181.61	-262.82	82.09	193.82	-325.20
Our (best)	**-232.45**	**-341.76**	**-195.50**	**-211.12**	**-140.78**	**-131.32**	**-277.02**	**-241.25**	**-189.06**	**-327.81**
Our (worst)	-229.80	**-341.76**	-190.47	-187.11	-83.55	-111.16	**-277.02**	-189.33	-124.03	**-327.81**
Our-C (best)	-230.63	**-341.76**	-191.28	-203.14	-132.29	-120.09	**-277.02**	-225.48	-187.64	**-327.81**
Our-C (worst)	-141.24	-334.44	-122.75	-146.23	-59.94	-84.67	-227.96	-135.49	-83.07	-274.08
PPI ($\alpha = -1$) [14]	1778.67	1739.01	2009.06	1933.65	2160.84	2257.74	1933.82	1652.54	1896.90	1941.30
PPI ($\alpha = -10$) [14]	1778.67	1739.01	2009.06	1933.65	2160.84	2257.74	1933.82	1652.54	1896.90	1941.30
PPI ($\alpha = -50$) [14]	1778.67	1739.01	2009.06	1933.65	2160.84	2257.74	1933.82	1652.54	1896.90	1941.30
PPI ($\alpha = 0$) [14]	1778.67	1739.01	2009.06	1933.65	2160.84	2257.74	1933.82	1652.54	1896.90	1941.30
Stiefel [7]	1997.70	1680.36	2253.16	2288.69	2607.97	2622.23	1916.57	1741.68	1813.59	1616.33
Spectral [36, 9]	1276.50	1006.73	1175.67	1642.52	1908.62	1778.91	1142.10	1180.81	1320.92	728.80

Table E.20: **Hotel (16 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	1068.17	999.03	1054.45	1569.10	1596.49	895.98	693.12	138.39	1371.56	573.36
MP-T [44]	-186.95	**-608.31**	-364.29	-376.98	-248.73	-206.48	-495.75	-490.30	-205.27	**-603.05**
MST [45]	-146.31	-559.87	-339.27	-296.91	-99.88	-93.65	-443.38	-403.01	-100.37	-533.47
MST-Star [52]	-271.32	-527.84	-318.06	-294.79	-137.19	-155.18	-394.38	-395.85	-164.61	-495.69
Our (Syn)	365.35	-602.05	265.51	207.55	909.96	541.86	-319.49	-37.88	549.89	-377.77
Our (best)	**-377.28**	**-608.31**	**-390.82**	**-396.30**	**-261.07**	**-285.30**	**-501.24**	**-495.07**	**-292.80**	**-603.05**
Our (worst)	-360.91	-607.96	-368.46	-339.09	-179.45	-225.34	-475.42	-475.32	-189.07	**-603.05**
Our-C (best)	-364.89	**-608.31**	-372.96	-383.80	-221.75	-274.43	-483.31	-469.22	-255.85	**-603.05**
Our-C (worst)	-229.88	-547.66	-325.79	-292.44	-107.99	-174.33	-408.72	-378.82	-130.73	-569.49
PPI ($\alpha = -1$) [14]	3190.40	2938.13	3464.07	3583.64	3517.20	4243.95	3087.30	2851.73	3628.93	3023.43
PPI ($\alpha = -10$) [14]	3190.40	2938.13	3464.07	3583.64	3517.20	4243.95	3087.30	2851.73	3628.93	3023.43
PPI ($\alpha = -50$) [14]	3190.40	2938.13	3464.07	3583.64	3517.20	4243.95	3087.30	2851.73	3628.93	3023.43
PPI ($\alpha = 0$) [14]	3190.40	2938.13	3464.07	3583.64	3517.20	4243.95	3087.30	2851.73	3628.93	3023.43
Stiefel [7]	3909.72	3129.24	3478.68	3784.96	4647.44	4764.61	3095.22	2957.87	3712.68	3014.59
Spectral [36, 9]	2294.19	2333.57	2282.51	1666.33	3180.32	2690.89	1937.30	1019.42	2048.70	1992.59

Table E.21: **House (4 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	57.43	125.16	131.14	10.22	152.14	58.07	150.27	57.74	105.42	92.02
MP-T [44]	-31.07	-19.17	**-33.91**	**-32.64**	-11.09	**-25.03**	**-24.46**	**-40.35**	-19.34	-12.63
MST [45]	-31.07	-20.21	-33.17	-31.77	-11.16	-21.74	**-24.46**	-39.65	-15.86	**-15.90**
MST-Star [52]	-31.07	-20.21	-33.17	-28.93	-9.19	-21.13	**-24.46**	-39.65	-14.39	**-15.90**
Our (Syn)	-29.99	-12.88	-32.69	-31.01	21.75	**-25.03**	-0.63	**-40.35**	-8.40	13.27
Our (best)	**-31.18**	**-20.40**	**-33.91**	**-32.64**	**-13.36**	**-25.03**	**-24.46**	**-40.35**	**-23.09**	**-15.90**
Our (worst)	**-31.18**	-19.95	**-33.91**	**-32.64**	-8.86	**-25.03**	**-24.46**	**-40.35**	**-23.09**	**-15.90**
Our-C (best)	**-31.18**	**-20.40**	**-33.91**	**-32.64**	**-13.36**	**-25.03**	**-24.46**	**-40.35**	**-23.09**	**-15.90**
Our-C (worst)	-28.33	-18.42	-31.63	-25.05	-8.68	-20.98	-21.11	-37.61	-15.64	-13.23
PPI ($\alpha = -1$) [14]	136.23	125.16	180.12	92.32	251.83	177.24	180.57	207.87	197.94	246.69
PPI ($\alpha = -10$) [14]	136.23	125.16	180.12	92.32	251.83	177.24	180.57	207.87	197.94	246.69
PPI ($\alpha = -50$) [14]	136.23	125.16	180.12	92.32	251.83	177.24	180.57	207.87	197.94	246.69
PPI ($\alpha = 0$) [14]	136.23	125.16	180.12	92.32	251.83	177.24	180.57	207.87	197.94	246.69
Stiefel [7]	137.69	166.61	119.83	124.77	198.25	73.96	172.71	127.88	211.53	192.28
Spectral [36, 9]	86.19	70.10	29.19	83.02	229.73	96.58	105.49	71.50	151.85	126.87

Table E.22: **House (8 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	157.73	169.53	282.35	159.88	457.37	215.69	454.89	333.47	206.87	261.92
MP-T [44]	-146.75	**-145.68**	-101.38	-112.32	**-102.53**	-144.82	-89.98	-116.46	-132.30	-79.20
MST [45]	-114.60	-134.33	-96.38	-105.50	-97.23	-126.63	-83.42	-102.53	-105.37	-70.95
MST-Star [52]	-134.26	-129.32	-88.49	-104.37	-83.84	-128.46	-78.53	-100.60	-94.23	-75.18
Our (Syn)	-102.39	-127.82	50.60	9.11	-88.54	-141.64	-1.69	-104.82	-107.90	-72.70
Our (best)	**-147.52**	**-145.68**	**-113.22**	**-116.08**	**-102.53**	**-145.90**	**-93.64**	**-116.73**	**-134.16**	**-89.10**
Our (worst)	-146.75	-143.81	-102.60	-106.78	-95.21	-137.45	**-93.64**	-113.78	**-134.16**	-74.07
Our-C (best)	**-147.52**	-144.15	-113.14	**-116.08**	-101.92	-145.37	**-93.64**	-115.83	**-134.16**	-85.23
Our-C (worst)	-136.40	-129.43	-96.37	-103.74	-51.77	-128.21	-68.64	-84.97	-91.82	-68.64
PPI ($\alpha = -1$) [14]	584.87	607.67	849.19	631.56	763.97	645.70	930.20	806.16	618.87	704.57
PPI ($\alpha = -10$) [14]	584.87	607.67	849.19	631.56	763.97	645.70	930.20	806.16	618.87	704.57
PPI ($\alpha = -50$) [14]	584.87	607.67	849.19	631.56	763.97	645.70	930.20	806.16	618.87	704.57
PPI ($\alpha = 0$) [14]	584.87	607.67	849.19	631.56	763.97	645.70	930.20	806.16	618.87	704.57
Stiefel [7]	573.68	601.70	766.95	586.92	745.64	645.58	945.36	722.57	650.21	978.02
Spectral [36, 9]	263.27	234.54	419.22	490.95	190.60	279.34	707.91	419.61	421.87	378.20

Table E.23: **House (12 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	426.65	261.60	156.20	900.55	907.48	165.56	745.28	741.60	101.61	478.88
MP-T [44]	**-363.43**	-390.65	-249.22	-246.86	-231.75	-351.03	-228.90	**-355.49**	-277.57	-198.67
MST [45]	-289.71	-330.79	-173.15	-217.05	-204.84	-286.46	-193.13	-327.71	-250.49	-176.06
MST-Star [52]	-322.89	-324.23	-185.50	-215.73	-190.46	-313.64	-194.31	-302.50	-222.95	-176.63
Our (Syn)	-253.95	-358.46	-84.77	-79.20	-21.31	-323.76	-186.99	-323.69	-176.48	-155.42
Our (best)	**-363.43**	**-391.51**	**-260.92**	**-249.83**	**-236.53**	**-353.46**	**-257.68**	**-355.49**	**-283.75**	**-223.04**
Our (worst)	**-363.43**	**-391.51**	**-260.92**	-220.06	-216.38	-352.24	-248.19	**-355.49**	-165.53	-194.62
Our-C (best)	**-363.43**	-390.92	-246.56	-242.82	-231.12	-349.11	-242.73	**-355.49**	-282.67	-213.06
Our-C (worst)	-159.87	-342.97	-143.01	-170.58	-111.24	-319.59	-201.58	-268.65	-143.74	-130.61
PPI ($\alpha = -1$) [14]	1319.68	1342.21	1634.18	1640.06	1968.44	1460.50	1814.31	1470.12	1392.24	2063.46
PPI ($\alpha = -10$) [14]	1319.68	1342.21	1634.18	1640.06	1968.44	1460.50	1814.31	1470.12	1392.24	2063.46
PPI ($\alpha = -50$) [14]	1319.68	1342.21	1634.18	1640.06	1968.44	1460.50	1814.31	1470.12	1392.24	2063.46
PPI ($\alpha = 0$) [14]	1319.68	1342.21	1634.18	1640.06	1968.44	1460.50	1814.31	1470.12	1392.24	2063.46
Stiefel [7]	1350.64	1353.99	1437.91	1730.19	2224.61	1377.13	1878.36	1429.20	1318.21	2157.14
Spectral [36, 9]	496.81	753.09	495.64	910.74	1066.84	466.91	1749.49	975.07	610.82	1279.84

Table E.24: **House (16 objects)**. Objective values per instance.

	Instance									
Method	1	2	3	4	5	6	7	8	9	10
MP-T (Syn)	319.66	705.93	642.33	901.99	853.03	235.58	734.33	1152.72	391.73	915.33
MP-T [44]	-667.38	**-715.49**	-486.26	-469.58	-451.45	**-649.10**	-440.82	**-638.35**	-392.39	-397.59
MST [45]	-536.05	-559.74	-415.77	-417.18	-393.44	-520.41	-351.99	-559.70	-309.78	-345.32
MST-Star [52]	-573.66	-567.98	-347.24	-403.03	-336.81	-572.01	-337.31	-532.08	-280.27	-347.85
Our (Syn)	-526.74	-656.03	-206.44	-164.06	115.25	-579.26	-256.71	-596.50	-62.87	-260.43
Our (best)	**-667.56**	**-715.49**	**-502.76**	**-489.41**	**-479.29**	**-649.10**	**-480.40**	**-638.35**	**-441.27**	**-443.21**
Our (worst)	-665.67	-714.79	-462.39	-423.45	-419.96	-643.48	-460.30	-630.82	-356.24	-324.86
Our-C (best)	-659.33	-709.27	-491.07	-479.01	-476.99	-633.75	-454.16	-637.70	-396.78	-416.62
Our-C (worst)	-571.72	-664.26	-227.00	-325.90	-270.26	-558.03	-321.19	-563.36	-285.76	-238.97
PPI ($\alpha = -1$) [14]	2412.77	2263.70	2553.10	2806.94	3280.38	2491.27	2836.06	2891.44	3259.36	3199.15
PPI ($\alpha = -10$) [14]	2412.77	2263.70	2553.10	2806.94	3280.38	2491.27	2836.06	2891.44	3259.36	3199.15
PPI ($\alpha = -50$) [14]	2412.77	2263.70	2553.10	2806.94	3280.38	2491.27	2836.06	2891.44	3259.36	3199.15
PPI ($\alpha = 0$) [14]	2412.77	2263.70	2553.10	2806.94	3280.38	2491.27	2836.06	2891.44	3259.36	3199.15
Stiefel [7]	2365.88	2284.58	3148.88	2859.80	3700.07	2539.10	2753.59	2758.95	3088.10	3436.32
Spectral [36, 9]	1049.60	1372.92	1969.43	1776.11	2113.20	1096.22	1711.94	1770.04	1466.61	2185.79

Table E.25: **Worms (3 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-211016.67	-208156.26	-203697.82	-196668.37	-192706.17	-193930.58	-208327.26	-198582.59	-216768.12	-205408.33
MP-T [44]	-177433.51	-173780.52	-178503.66	-169398.37	-169689.28	-167291.05	-177953.84	-170321.01	-175987.63	-168465.39
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-213872.86	-211511.08	-208820.70	-204001.19	-201206.96	-203750.85	-211816.97	-206455.20	-218452.77	-211726.52
Our (best)	**-217522.65**	**-215454.88**	**-215199.17**	**-210777.35**	**-210598.58**	**-210471.70**	**-215829.86**	**-212856.99**	**-221534.89**	**-215555.11**
Our (worst)	-217174.97	-215340.78	-214761.36	-210394.74	-210261.78	-210214.41	-215578.46	-212215.16	-221284.18	-215273.66
Our-C (best)	-216737.76	-214834.72	-214259.10	-209395.67	-209642.54	-209430.28	-215358.17	-211806.00	-221186.58	-214813.92
Our-C (worst)	-216316.70	-213922.96	-213087.39	-207506.61	-207791.70	-208384.29	-214206.09	-209991.58	-220331.84	-214093.14
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.26: **Worms (4 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-416908.40	-397253.22	-404088.74	-402283.40	-413673.35	-420661.19	-415624.51	-403705.18	-414972.92	-409729.08
MP-T [44]	-342771.63	-332156.36	-330145.63	-335375.92	-341906.33	-338015.52	-340481.69	-331096.25	-335178.64	-348453.65
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-419378.13	-403620.27	-407458.64	-404144.00	-417806.74	-420905.35	-418848.96	-412465.50	-418328.31	-412238.92
Our (best)	**-432492.80**	**-423476.43**	**-425229.64**	**-422593.38**	**-431920.67**	**-432793.85**	**-432705.38**	**-426378.82**	**-432107.82**	**-430375.59**
Our (worst)	-432073.67	-421898.06	-423420.34	-420989.75	-430182.44	-432363.66	-432086.18	-425606.18	-431079.91	-428961.96
Our-C (best)	-430647.09	-421341.83	-422762.46	-419303.03	-429888.09	-431220.07	-430554.94	-423626.22	-430758.66	-427982.19
Our-C (worst)	-427726.22	-416653.33	-419032.44	-417471.66	-427884.47	-427672.21	-428571.72	-418944.09	-427830.61	-424674.73
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.27: **Worms (5 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-686421.76	-673181.30	-696359.64	-692409.29	-687994.30	-694637.75	-676107.84	-694504.92	-689903.58	-689757.17
MP-T [44]	-556779.60	-552338.16	-557822.95	-549829.07	-552029.53	-553601.42	-548888.82	-555146.05	-560515.26	-549732.95
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-696705.57	-680184.63	-703951.51	-697875.74	-693885.93	-699523.98	-688168.37	-700481.06	-696476.73	-696252.38
Our (best)	**-719704.60**	**-708814.31**	**-718663.16**	**-717930.04**	**-716916.33**	**-717935.18**	**-711323.30**	**-717136.96**	**-717023.66**	**-716040.58**
Our (worst)	-717091.92	-706369.95	-717586.67	-715652.13	-714795.00	-716531.28	-710229.48	-714687.79	-715276.26	-714827.11
Our-C (best)	-714792.05	-704546.87	-715085.48	-713518.53	-711539.26	-714801.03	-706867.91	-712212.62	-712143.43	-713571.86
Our-C (worst)	-710415.98	-694913.56	-709632.08	-707393.05	-708157.42	-708991.37	-699549.48	-707875.75	-706455.41	-706458.75
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.28: **Worms (6 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-1021192.31	-1041112.30	-1035985.09	-1042334.68	-1036744.76	-1032838.51	-1040724.42	-1041100.22	-1039420.26	-1037111.87
MP-T [44]	-817795.98	-823492.65	-820057.88	-812306.05	-824559.41	-826352.93	-810666.26	-811507.99	-826202.06	-815589.18
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-1035338.50	-1049818.24	-1039602.87	-1048252.84	-1050971.63	-1030510.69	-1044467.14	-1046649.15	-1045187.01	-1043295.35
Our (best)	**-1067304.57**	**-1077004.01**	**-1075811.41**	**-1076387.19**	**-1074800.27**	**-1064503.77**	**-1073901.60**	**-1074173.43**	**-1073257.14**	**-1071755.00**
Our (worst)	-1063079.91	-1074564.62	-1072893.48	-1072947.77	-1072166.69	-1060482.88	-1070949.03	-1070075.01	-1070690.52	-1068628.77
Our-C (best)	-1059720.15	-1068549.92	-1066918.03	-1068388.95	-1066873.23	-1056711.64	-1065051.97	-1066236.98	-1066991.58	-1061486.98
Our-C (worst)	-1043200.40	-1062631.30	-1059814.54	-1060413.68	-1058084.06	-1048126.78	-1055387.50	-1056446.79	-1060124.02	-1057892.91
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.29: **Worms (7 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-1445401.74	-1436506.34	-1451390.28	-1447214.69	-1454372.16	-1461535.27	-1429469.33	-1450613.18	-1463509.36	-1453445.69
MP-T [44]	-1145530.78	-1112774.86	-1131316.02	-1144590.80	-1155885.55	-1132631.56	-1111282.47	-1125492.38	-1130527.77	-1142180.35
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-1456311.40	-1447199.83	-1467741.66	-1464540.93	-1463225.41	-1469313.04	-1445586.85	-1462464.90	-1470458.11	-1461995.62
Our (best)	**-1493077.98**	**-1494048.49**	**-1503654.25**	**-1501596.32**	**-1502135.93**	**-1505313.81**	**-1490538.17**	**-1503215.68**	**-1503014.88**	**-1500384.80**
Our (worst)	-1488654.88	-1487775.72	-1499078.04	-1497588.08	-1499376.00	-1498845.55	-1486296.94	-1498616.65	-1498512.79	-1496526.31
Our-C (best)	-1481409.09	-1482692.62	-1489699.43	-1486764.21	-1487636.05	-1490226.47	-1474078.87	-1490370.49	-1490304.31	-1490768.53
Our-C (worst)	-1458616.09	-1472658.34	-1479293.72	-1479554.24	-1480319.13	-1482641.75	-1452803.09	-1479271.55	-1478941.71	-1479651.13
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.30: **Worms (8 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-1942140.72	-1926255.91	-1926672.62	-1941028.40	-1936929.55	-1935015.67	-1935780.40	-1930411.12	-1933985.77	-1925898.65
MP-T [44]	-1503720.31	-1494037.63	-1497606.43	-1490221.38	-1486414.94	-1490000.28	-1503775.54	-1474272.93	-1519216.32	-1520142.14
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-1957450.62	-1939620.54	-1935423.05	-1950881.12	-1948860.69	-1943401.59	-1945747.74	-1942247.90	-1945881.88	-1946556.52
Our (best)	**-1999990.91**	**-1986823.36**	**-1986189.67**	**-1999684.79**	**-1999285.29**	**-1996646.15**	**-1998268.17**	**-1993949.62**	**-1996852.07**	**-1997574.28**
Our (worst)	-1992823.58	-1984938.43	-1979986.23	-1994474.11	-1991122.07	-1984644.28	-1991278.38	-1988300.54	-1989832.26	-1991442.98
Our-C (best)	-1983115.67	-1969740.36	-1963576.49	-1980485.48	-1975734.54	-1973229.56	-1975999.15	-1976667.68	-1975758.62	-1981135.85
Our-C (worst)	-1963785.96	-1958977.97	-1937320.72	-1968778.22	-1966493.99	-1962392.04	-1963083.38	-1966771.17	-1962385.20	-1960000.60
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.31: **Worms (9 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-2504985.81	-2492764.37	-2496341.13	-2495957.45	-2512705.94	-2484750.27	-2514464.54	-2486363.62	-2513359.25	-2485034.69
MP-T [44]	-1922118.45	-1959860.94	-1923370.31	-1895073.04	-1887813.20	-1902582.11	-1942511.58	-1929583.00	-1949715.28	-1937657.25
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-2511543.15	-2505363.44	-2511015.17	-2518954.55	-2524049.24	-2502178.07	-2522208.80	-2497298.54	-2517261.89	-2495632.85
Our (best)	**-2566492.95**	**-2565434.05**	**-2563374.38**	**-2564475.34**	**-2581582.12**	**-2559492.63**	**-2582885.43**	**-2560993.89**	**-2583695.60**	**-2559715.21**
Our (worst)	-2555124.31	-2555821.16	-2553940.96	-2554864.99	-2574644.72	-2553518.01	-2575748.07	-2550273.58	-2576919.87	-2550658.06
Our-C (best)	-2536758.10	-2540212.39	-2541863.25	-2532166.33	-2553265.78	-2530686.73	-2552016.58	-2528785.26	-2556311.05	-2534129.29
Our-C (worst)	-2519708.64	-2524335.17	-2515474.85	-2516729.16	-2535255.07	-2505732.98	-2532253.95	-2512300.14	-2544425.96	-2513488.69
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.32: **Worms (10 objects)**. Objective values per instance.

	Instance									
Method	0	1	2	3	4	5	6	7	8	9
MP-T (Syn)	-3131272.87	-3113149.83	-3119535.32	-3111113.68	-3103769.49	-3131376.71	-3118397.46	-3130512.79	-3112547.40	-3097622.06
MP-T [44]	-2396051.47	-2399287.91	-2384910.77	-2418170.64	-2379910.84	-2389996.36	-2371701.10	-2437000.82	-2351875.27	-2401302.08
MST [45]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
MST-Star [52]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Our (Syn)	-3149633.12	-3130488.24	-3131681.69	-3130616.76	-3115485.04	-3145352.51	-3129153.47	-3139715.68	-3129906.26	-3123734.50
Our (best)	**-3220291.02**	**-3200220.09**	**-3201979.16**	**-3198230.71**	**-3197242.83**	**-3215285.84**	**-3197181.82**	**-3216187.68**	**-3194085.42**	**-3197234.58**
Our (worst)	-3206480.07	-3191007.10	-3189750.06	-3188921.13	-3190156.39	-3206885.10	-3187830.10	-3201529.71	-3188905.18	-3184930.80
Our-C (best)	-3187281.05	-3159168.93	-3166865.99	-3160159.50	-3165609.07	-3183557.18	-3158197.61	-3180563.37	-3155481.63	-3161826.38
Our-C (worst)	-3159966.43	-3145862.89	-3132114.22	-3145457.87	-3140330.42	-3162280.05	-3140378.97	-3153200.24	-3133200.50	-3126978.40
PPI ($\alpha = -1$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -10$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = -50$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
PPI ($\alpha = 0$) [14]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Stiefel [7]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf
Spectral [36, 9]	inf	inf	inf	inf	inf	inf	inf	inf	inf	inf

Table E.33: **Worms (29 objects)**. Objective values per instance.

	Instance									
Method	0	-	-	-	-	-	-	-	-	-
MP-T (Syn)	-26135912.49	-	-	-	-	-	-	-	-	-
MP-T [44]	-21748249.19	-	-	-	-	-	-	-	-	-
MST [45]	inf	-	-	-	-	-	-	-	-	-
MST-Star [52]	inf	-	-	-	-	-	-	-	-	-
Our (Syn)	-26342685.93	-	-	-	-	-	-	-	-	-
Our (best)	**-27115925.70**	-	-	-	-	-	-	-	-	-
Our (worst)	-26947041.10	-	-	-	-	-	-	-	-	-
Our-C (best)	-26674999.96	-	-	-	-	-	-	-	-	-
Our-C (worst)	-26232974.36	-	-	-	-	-	-	-	-	-
PPI ($\alpha = -1$) [14]	inf	-	-	-	-	-	-	-	-	-
PPI ($\alpha = -10$) [14]	inf	-	-	-	-	-	-	-	-	-
PPI ($\alpha = -50$) [14]	inf	-	-	-	-	-	-	-	-	-
PPI ($\alpha = 0$) [14]	inf	-	-	-	-	-	-	-	-	-
Stiefel [7]	inf	-	-	-	-	-	-	-	-	-
Spectral [36, 9]	inf	-	-	-	-	-	-	-	-	-